

Program 1: Installing Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup

Java (JDK) Installation:

Step 1: Type the command **sudo apt install openjdk-11-jdk -y** in the terminal

It will install the OpenJDK 17 automatically.

```
saniksha@saniksha:~$ sudo apt install openjdk-11-jdk
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openjdk-11-jdk is already the newest version (11.0.30+7-1ubuntu1-22.04).
The following packages were automatically installed and are no longer required:
  linux-headers-6.8.0-40-generic linux-hwe-6.8-headers-6.8.0-40 linux-hwe-6.8-tools-6.8.0-40 linux-image-6.8.0-40-generic
  linux-modules-6.8.0-40-generic linux-modules-extra-6.8.0-40-generic linux-tools-6.8.0-40-generic
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 44 not upgraded.
saniksha@saniksha:~$
```

Step 2: Installation can be confirmed by running the command **java -version**

```
saniksha@saniksha:~$ java --version
openjdk 17.0.18 2020-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-122.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-122.04.1, mixed mode, sharing)
saniksha@saniksha:~$
```

Maven Installation:

Step 1: Type the command **sudo apt install maven** in the terminal.

It will install Apache Maven from Ubuntu's package repository.

```
saniksha@saniksha:~$ sudo apt install maven
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
maven is already the newest version (3.6.3-5).
The following packages were automatically installed and are no longer required:
  linux-headers-6.8.0-40-generic linux-hwe-6.8-headers-6.8.0-40 linux-hwe-6.8-tools-6.8.0-40 linux-image-6.8.0-40-generic
  linux-modules-6.8.0-40-generic linux-modules-extra-6.8.0-40-generic linux-tools-6.8.0-40-generic
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 44 not upgraded.
saniksha@saniksha:~$
```

Step 2: Installation can be confirmed by running the command **mvn -version**

```
santiksha@santiksha:~$ mvn --version
Apache Maven 3.6.3
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Ubuntu, runtime: /usr/lib/jvm/java-17-openjdk-amd64
Default locale: en_IN, platform encoding: UTF-8
OS name: "linux", version: "6.8.0-101-generic", arch: "amd64", family: "unix"
santiksha@santiksha:~$
```

Gradle Installation:

Step 1: Type the command **sudo apt install gradle** in the terminal

It installs Gradle build automation tool from Ubuntu's package repository.

```
santiksha@santiksha:~$ sudo apt install gradle
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gradle is already the newest version (4.4.1-13).
The following packages were automatically installed and are no longer required:
  linux-headers-6.8.0-40-generic linux-hwe-6.8-headers-6.8.0-40 linux-hwe-6.8-tools-6.8.0-40 linux-image-6.8.0-40-generic
  linux-modules-6.8.0-40-generic linux-modules-extra-6.8.0-40-generic linux-tools-6.8.0-40-generic
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 44 not upgraded.
santiksha@santiksha:~$
```

Step 2: To check the proper installation of Gradle run the command **gradle --version**

```
santiksha@santiksha:~$ gradle --version

-----
Gradle 4.4.1
-----

Build time:   2012-12-21 00:00:00 UTC
Revision:     none

Groovy:       2.4.21
Ant:          Apache Ant(TM) version 1.10.12 compiled on January 17 1978
JVM:          17.0.18 (Ubuntu 17.0.18-8-Ubuntu-122.04.1)
OS:           Linux 6.8.0-101-generic amd64
```

Program 2: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Step 1: To create a Maven project run the following command in terminal:

**mvn archetype:generate -DgroupId=com.example -
DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-
quickstart -DinteractiveMode=false**

This command generates a basic Maven project structure with a sample App.java and AppTest.java.

```

sanikshagsaniksha: $ mkdir report
sanikshagsaniksha: $ cd report
sanikshagsaniksha: ~/$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
-----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
>>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
<<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
--- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 MB at 5.8 MB/s)
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/saniksha/report
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/saniksha/report/MyMavenApp
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 13.271 s
[INFO] Finished at: 2026-03-17T18:56:11+05:30
[INFO]
sanikshagsaniksha: ~/$

```

This “BUILD SUCCESS” will confirm that the project is created successfully

Step 2: Go to the project directory using command **cd MyMavenApp**

Step 3: Open and edit pom.xml file using command **gedit pom.xml**

- Go to <https://mvnrepository.com>
- Search for a dependency “junit”
- Select the latest version and copy the dependency XML snippet.
- Paste it under the <dependencies> section in pom.xml
- Updated pom.xml:

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0

```

```

http://maven.apache.org/maven-v4_0_0.xsd">

```

```

<modelVersion>4.0.0</modelVersion>

```

```

<groupId>com.example</groupId>

```

```

<artifactId>MyMavenApp</artifactId>

```

```

<version>1.0-SNAPSHOT</version>

```

```

<packaging>jar</packaging>

```

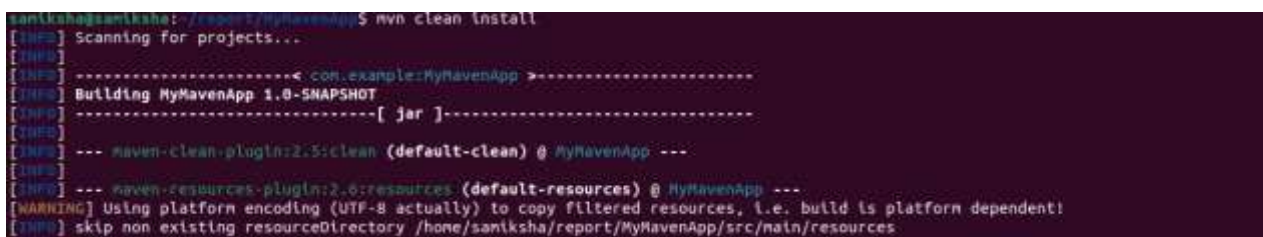
```
<name>MyMavenApp</name>
<url>http://maven.apache.org</url>
<!-- Java version configuration -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
<!-- Dependencies -->
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
</dependencies>
<!-- Build Plugins -->
<build>
<plugins>
<!-- Compiler Plugin -->
<plugin>
<groupId>org.apache.maven.plugins</groupId> <artifactId>maven-
compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>11</source>
<target>11</target>
</configuration>
```

```

</plugin>
<!-- Surefire Plugin for Testing -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<!-- JAR Plugin with Entry Point -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<version>3.1.0</version>
<configuration>
<archive>
<manifestEntries>
<Main-Class>com.example.App</Main-Class>
</manifestEntries>
</archive>
</configuration>
</plugin>
</plugins>
</build>
</project>

```

Step 4: Build the project using command **mvn clean install**



```

sanikshasamiksha: /home/saniksha/MyHavenApp$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyHavenApp >-----
[INFO] Building MyHavenApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyHavenApp ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyHavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/saniksha/report/MyHavenApp/src/main/resources

```

```
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenApp ---
[INFO] Building jar: /home/saniksha/report/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar
[INFO] --- maven-install-plugin:2.4.1:install (default-install) @ MyMavenApp ---
[INFO] Installing /home/saniksha/report/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar to /home/saniksha/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/saniksha/report/MyMavenApp/pom.xml to /home/saniksha/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.566 s
[INFO] Finished at: 2026-03-17T19:08:24+05:30
[INFO] -----
```

The “Build Success” message ensures that our Maven project is built successfully.

Step 5: To run the Maven application type the command:

java -jar target/MyMavenApp-1.0-SNAPSHOT.jar

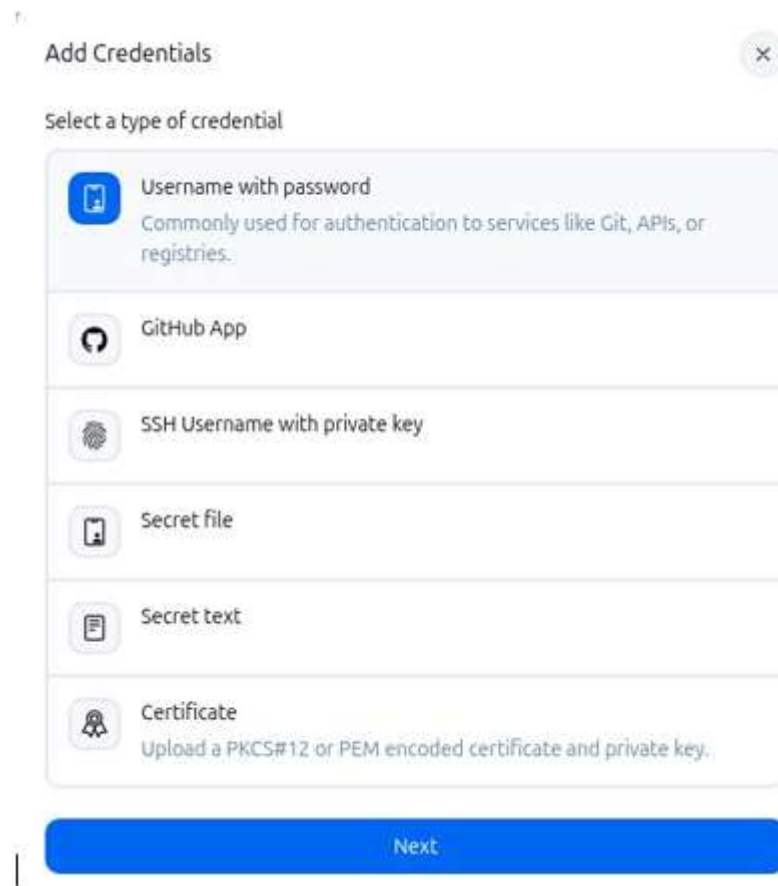
- After completing the execution, the output will look like:

```
sanikshagsaniksha:~/report/MyMavenApp$ java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
Hello World!
sanikshagsaniksha:~/report/MyMavenApp$
```

This will show output as the code present in App.java file we can edit the text to get any other output.

Step 6: Login to Jenkins and go to the Credentials page and click on add credentials

Step 7: Add new credentials in Jenkins for CI pipeline. Enter details like username, password, set the scope to Global, and provide a description.



Step 8: now open the terminal and type the command **cd MyMavenApp** to enter the folder and create a **Jenkinsfile**. Paste the code in the Jenkinsfile pipeline {

```
agent any // Use any available agent
```

```
tools {
```

```
    maven 'Maven' // Ensure this matches the name configured in Jenkins
```

```
}
```

```
stages {
```

```
    stage('Checkout') {
```

```
        steps {
```

```
            git
```

```
                branch:
```

```
                'master',
```

```
                url:
```

```
'https://github.com/Samiksha201104/MyMavenApp.git'
```

```
        }
```

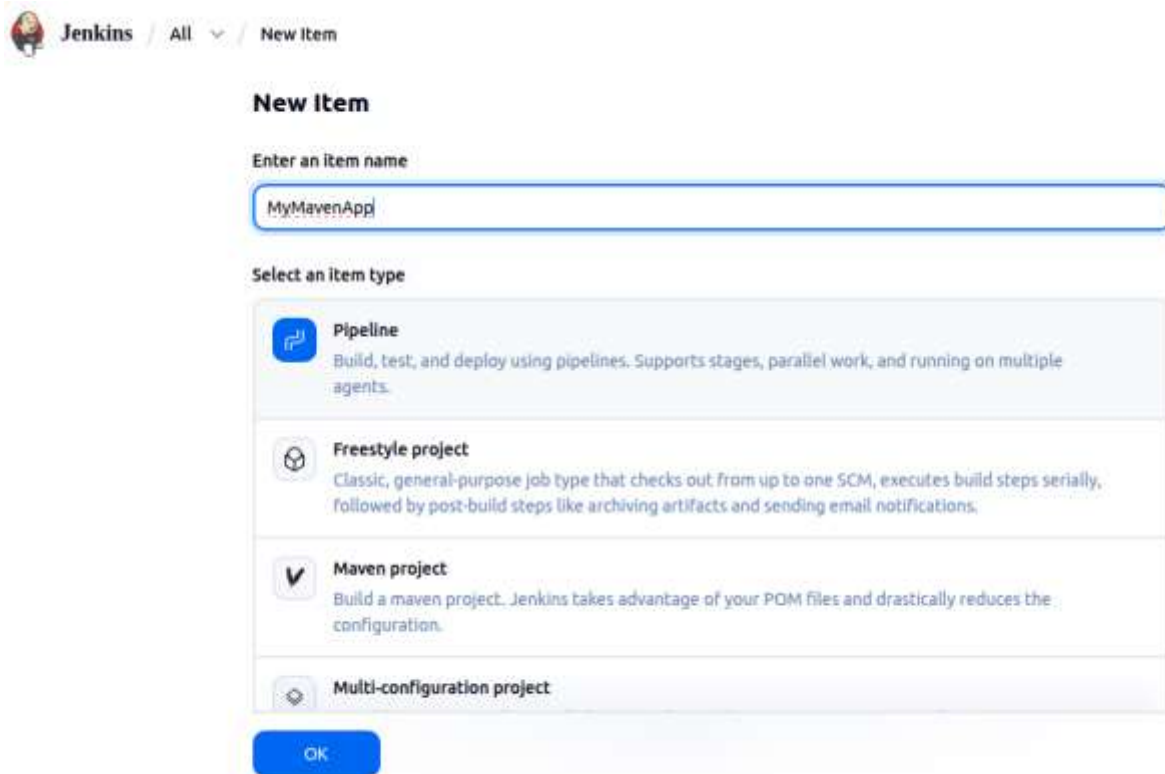


```
}  
stage('Build') {  
    steps {  
        sh 'mvn clean package' // Run Maven build  
    }  
}  
stage('Test') {  
    steps {  
        sh 'mvn test' // Run unit tests  
    }  
}  
stage('Run Application') {  
    steps {  
        // Start the JAR application  
        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'  
    }  
}  
}  
post {  
    success {  
        echo 'Build and deployment successful!'  
    }  
    failure {  
        echo 'Build failed!'  
    }  
}}
```

Then commit and push it to GitHub

```
samiksha@samiksha:~/Devops6D/MyMavenApp$ git init
Reinitialized existing Git repository in /home/samiksha/Devops6D/MyMavenApp/.git/
samiksha@samiksha:~/Devops6D/MyMavenApp$ git add .
samiksha@samiksha:~/Devops6D/MyMavenApp$ git commit -m "Adding Jenkinsfile"
[master 8735bc6] Adding Jenkinsfile
 1 file changed, 47 insertions(+)
 create mode 100644 Jenkinsfile
samiksha@samiksha:~/Devops6D/MyMavenApp$ git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 729 bytes | 729.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Samiksha201104/Devops6D.git
 c5925f2..8735bc6  master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
samiksha@samiksha:~/Devops6D/MyMavenApp$ █
```

Step 9: Go to Jenkins Dashboard and click on new item , enter the name as MyMavenApp and choose pipeline



Jenkins / All / New Item

New Item

Enter an item name

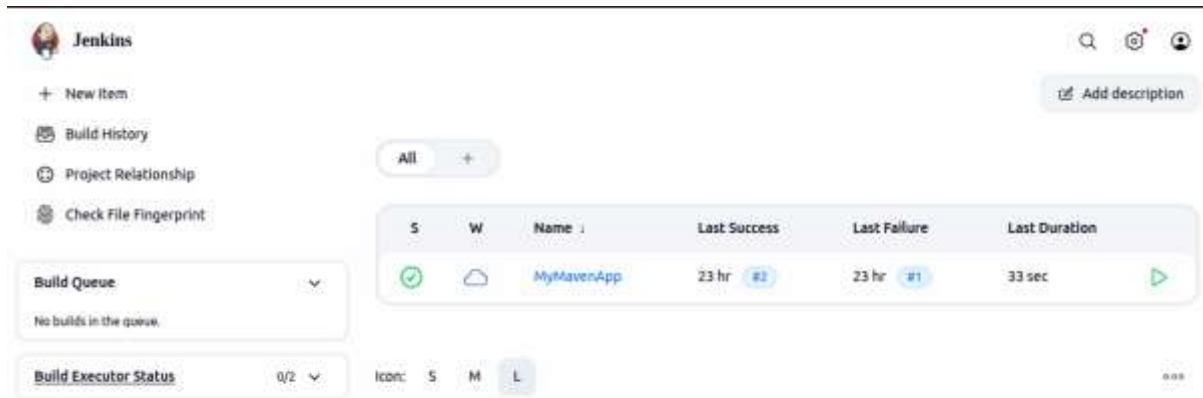
MyMavenApp

Select an item type

- ☐ Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- ☐ Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- ☒ Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- ☐ Multi-configuration project

OK

Step 10: in the Jenkins dashboard select MyMavenApp and select build now and then select full stage view



Step 11 : Note the stages that failed and check the error by clicking the stage and selecting logs.

MyMavenApp - Stage View



To correct the error open pom.xml and make changes in the code

The original code has the following lines

```
<source>1.7</source>
```

```
<target>1.7</target>
```

Change this code to

```
<source>11</source>
```

```
<target>11</target>
```

Push this back to GitHub and build it again on Jenkins

Creating a Maven Project with Guava and Commons-IO

Maven Project with Guava and Commons-IO

Dependencies Create a Maven Project Using

Terminal by following command:

mvn archetype: generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
sanikshagsaniksha: /reports$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/saniksha/reports
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenGuavaApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/saniksha/reports/MyMavenGuavaApp
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 5.360 s
[INFO] Finished at: 2020-03-17T19:29:09+05:30
[INFO]
[INFO] -----
sanikshagsaniksha: /reports$
```

Now Navigate to the Project Directory using **cd MyMavenGuavaApp**

```
saniksha@saniksha: /reports$ cd MyMavenGuavaApp
saniksha@saniksha: /reports/MyMavenGuavaApp$
```

Edit the pom.xml File and add the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenGuavaApp</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenGuavaApp</name>
  <url>http://maven.apache.org</url>
```

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>

  <!-- https://mvnrepository.com/artifact/com.google.guava/guava -->
<dependency>
  <groupId>com.google.guava</groupId>
  <artifactId>guava</artifactId>
  <version>33.4.0-jre</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
<dependency>
  <groupId>commons-io</groupId>
  <artifactId>commons-io</artifactId>
  <version>2.18.0</version>
</dependency>

</dependencies>

<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
```

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.1.0</version>
  <configuration>
    <archive>
      <manifestEntries>
        <Main-Class>com.example.App</Main-Class>
      </manifestEntries>
    </archive>
  </configuration>
</plugin>

</plugins>
</build>
</project>
```

Modify App.java

Got to App.java using **cd src/main/java/com/example/** and **gedit App.java** to type the file with following code:

```
package com.example;

/**
 * Hello world!
 *
 */

// Import FileUtils class from Apache Commons IO library
// This is used to perform file operations like copying files
import org.apache.commons.io.FileUtils;
```

```
// Import File class to represent file and directory pathnames
import java.io.File;

// Import IOException to handle input-output errors
import java.io.IOException;

// Import ImmutableList from Google Guava library
// ImmutableList creates a list that cannot be modified after creation
import com.google.common.collect.ImmutableList;

// Define the main class
public class App
{
    // Main method - program execution starts from here
    public static void main( String[] args )
    {
        // Create an immutable list of strings (fruits)
        // This list cannot be changed (no add/remove allowed)
        ImmutableList<String> fruits = ImmutableList.of("Apple", "Banana",
"Cherry");

        // Print the list of fruits to the console
        System.out.println(fruits);

        // Create a File object representing the source file
        File sourceFile = new File("source.txt");

        // Create a File object representing the destination file
        File destFile = new File("destination.txt");

        try {
```

```

// Copy the contents of source.txt to destination.txt
// Using Apache Commons IO FileUtils class
FileUtils.copyFile(sourceFile, destFile);

// Print success message if file is copied
System.out.println("File copied successfully!");
}
catch (IOException e) {
    // If any error occurs during file copying,
    // this block will handle the exception
    // Print error message
    System.err.println("Error occurred while copying file: " +
e.getMessage());
}
}
}
}

```

Build the Project using the command **mvn clean install** command

```

[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenGuavaApp ---
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.2.0/maven-archiver-3.2.0.pom (4.3 kB at 144 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.2.0/maven-archiver-3.2.0.jar (4.3 kB at 144 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.pom (4.9 kB at 189 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.jar (4.9 kB at 189 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.5/plexus-archiver-3.5.pom (4.8 kB at 185 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.5/plexus-archiver-3.5.jar (4.8 kB at 185 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/5.0/plexus-5.0.pom (21 kB at 790 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/5.0/plexus-5.0.jar (21 kB at 790 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.0.0/plexus-io-3.0.0.pom (4.5 kB at 175 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.0.0/plexus-io-3.0.0.jar (4.5 kB at 175 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.14/commons-compress-1.14.pom (13 kB at 505 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.14/commons-compress-1.14.jar (13 kB at 505 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/42/commons-parent-42.pom (68 kB at 2.2 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/42/commons-parent-42.jar (68 kB at 2.2 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/tukaani/xz/1.6/xz-1.6.pom (1.9 kB at 73 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/tukaani/xz/1.6/xz-1.6.jar (1.9 kB at 73 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.1.0/plexus-utils-3.1.0.pom (4.7 kB at 155 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.1.0/plexus-utils-3.1.0.jar (4.7 kB at 155 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.2.0/maven-archiver-3.2.0.jar (4.3 kB at 144 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.jar (4.9 kB at 189 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.5/plexus-archiver-3.5.jar (4.8 kB at 185 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.0.0/plexus-io-3.0.0.jar (4.5 kB at 175 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.14/commons-compress-1.14.jar (13 kB at 505 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.2.0/maven-archiver-3.2.0.jar (4.3 kB at 144 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/tukaani/xz/1.6/xz-1.6.jar (1.9 kB at 73 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.0.0/plexus-io-3.0.0.jar (4.5 kB at 175 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.1.0/plexus-utils-3.1.0.jar (4.7 kB at 155 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.5/plexus-archiver-3.5.jar (4.8 kB at 185 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/42/commons-parent-42.jar (68 kB at 2.2 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.jar (4.9 kB at 189 kB/s)
[INFO] Building jar: /home/unified/Devops/MyMavenGuavaApp/target/MyMavenGuavaApp-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenGuavaApp ---
[INFO] Installing /home/unified/Devops/MyMavenGuavaApp/target/MyMavenGuavaApp-1.0-SNAPSHOT.jar to /home/unified/.m2/repository/com/example/MyMavenGuavaApp/1.0-SNAPSHOT/MyMavenGuavaApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/unified/Devops/MyMavenGuavaApp/pom.xml to /home/unified/.m2/repository/com/example/MyMavenGuavaApp/1.0-SNAPSHOT/MyMavenGuavaApp-1.0-SNAPSHOT.pom
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.003 s
[INFO] Finished at: 2025-05-28T21:31:52+05:30
[INFO]

```


This compiles the project and creates a JAR in the target/ folder.

Run the Application:

If java-jar command give an error which is most probable, use the following command **mvn exec:java -**

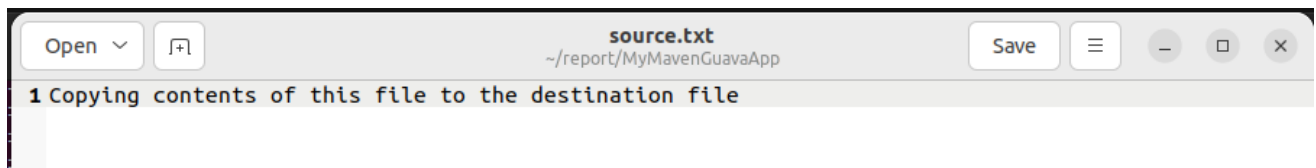
Dexec.mainClass="com.example.App"

```
saniksha@saniksha: ~/report/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass="com.example.App"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenGuavaApp >-----
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec-maven-plugin:3.0.3:java (default-cli) @ MyMavenGuavaApp ---
[Apple, Banana, Cherry]
Error occurred while copying file: Source 'source.txt' does not exist
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.559 s
[INFO] Finished at: 2026-03-17T20:28:33+05:30
[INFO]
```

This command gives error as

Error while copying file:Source 'source.txt' does not exist

Now, Create a Sample File to copy the content to destination file



Now Run the command **mvn exec:java -**

Dexec.mainClass="com.example.App" again

```
saniksha@saniksha: ~/report/MyMavenGuavaApp$ gedit source.txt
saniksha@saniksha: ~/report/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass="com.example.App"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenGuavaApp >-----
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec-maven-plugin:3.0.3:java (default-cli) @ MyMavenGuavaApp ---
[Apple, Banana, Cherry]
File copied successfully!
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.326 s
[INFO] Finished at: 2026-03-17T20:31:05+05:30
[INFO]
saniksha@saniksha: ~/report/MyMavenGuavaApp$
```

The output shows file copied successfully.

We can see newly created **destination.txt** using gedit destination.txt if you are still in Project's directory

```
saniksha@saniksha: ~/report/MyMavenGuavaApp$ ls
destination.txt pom.xml source.txt src target
saniksha@saniksha: ~/report/MyMavenGuavaApp$
```

File content is same as the content in source file.

Step 1: now open the terminal and type the command **cd MyMavenGuavaApp**

to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

pipeline {

agent any // Use any available agent

tools {

maven 'Maven' // Ensure this matches the name configured in Jenkins

}

stages {

stage('Checkout') {

steps {

git branch: 'master', url:

'https://github.com/Samiksha201104/MyMavenGuavaApp.git'

}

}

stage('Build') {

steps {

sh 'mvn clean package' // Run Maven build

}

}

stage('Test') {

steps {

sh 'mvn test' // Run unit tests

}

}

stage('Run Application') {

steps {

// Start the JAR application

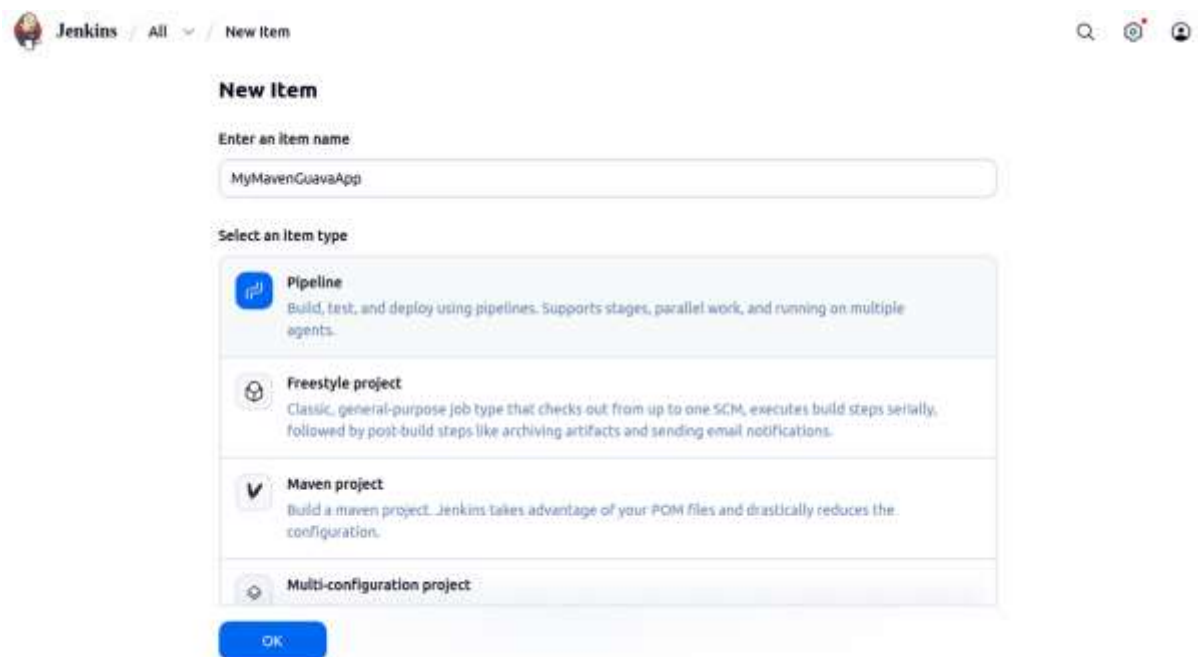
sh 'java -jar target/MyMavenGuavaApp-1.0-SNAPSHOT.jar'

}

```
}  
}  
post {  
    success {  
        echo 'Build and deployment successful!'  
    }  
    failure {  
        echo 'Build failed!'  
    }  
}}
```

Then commit and push it to GitHub

Step 2: in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials

Configure

General
Triggers
Pipeline
Advanced

Pipeline script from SCM

SCM
Git

Repositories

Repository URL
https://github.com/Saniksha201104/MyMavenGuavaApp.git
Please enter Git repository.

Credentials
GitHub Credentials
+ Add

Advanced

Save Apply

Step 3 : in the Dashboard click on the MyMavenGuavaApp item and select Build Now and then Full Stage view

Correct the errors and build again on Jenkins till all the stages are successful



Program 3: Working with Selenium: Maven Project To Connect To Web Servers Using Selenium Dependency

1.Sauce.demo webpage

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**

```
samiksha@samiksha:~/report$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/samiksha/report
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenSeleniumApp01
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/samiksha/report/MyMavenSeleniumApp01
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 4.456 s
[INFO] Finished at: 2026-03-17T20:40:07+05:30
[INFO]
```

Step 2: Before we connect to web servers we need to have chrome driver installed. To download chrome driver

To check the chrome version type the command **google-chrome --version** in the terminal

```
samiksha@samiksha:~/report$ google-chrome --version
Google Chrome 145.0.7632.116
samiksha@samiksha:~/report$
```

extract the version and store it in a variable **CHROME_VERSION**

```
samiksha@samiksha:~/report$ CHROME_VERSION=$(google-chrome --version | awk '{print $3}' | cut -d'.' -f1-3)
samiksha@samiksha:~/report$
```

extracts and transforms JSON data

```
LATEST_DRIVER=$(curl -s "https://googlechromelabs.github.io/chrome-for-testing/latest-patch-versions-per-build.json" | jq -r
".builds[\"$CHROME_VERSION\"].version")
```

do install curl using the command

```
sudo apt install curl
```

```
wget https://storage.googleapis.com/chrome-for-testing-public/$LATEST_DRIVER/linux64/chromedriver-linux64.zip
```

Extract and move it to /usr/local/bin/:

```
unzip chromedriver-linux64.zip
```

```
sudo mv chromedriver-linux64/chromedriver /usr/local/bin/chromedriver
```

```
sudo chmod +x /usr/local/bin/chromedriver
```

Check if ChromeDriver is installed correctly:

A terminal window with a dark purple background. The prompt is 'santkshagsantksha:~/project\$'. The command 'chromedriver --version' has been executed. The output is 'ChromeDriver 145.0.7632.117 (de4e5d6e13b995f5e04137976c0e8ec456017467-refs/branch-heads/7632@{#3101})'. The prompt is now 'santkshagsantksha:~/project\$' with a cursor.

```
santkshagsantksha:~/project$ chromedriver --version
ChromeDriver 145.0.7632.117 (de4e5d6e13b995f5e04137976c0e8ec456017467-refs/branch-heads/7632@{#3101})
santkshagsantksha:~/project$
```

Step 3 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenSeleniumApp01</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenSeleniumApp01</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
```

```
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
<!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java
-->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.29.0</version>
</dependency>
</dependencies>
<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.1</version>
  <configuration>
    <source>1.7</source>
    <target>1.7</target>
  </configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
```

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-jar-plugin</artifactId>
    <version>3.0.1</version>
    <configuration>
        <archive>
            <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
        </archive>
    </configuration>

</plugin>

</plugins>
</build>
</project>
```

Step 4: Edit the App.java

```
package com.example;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

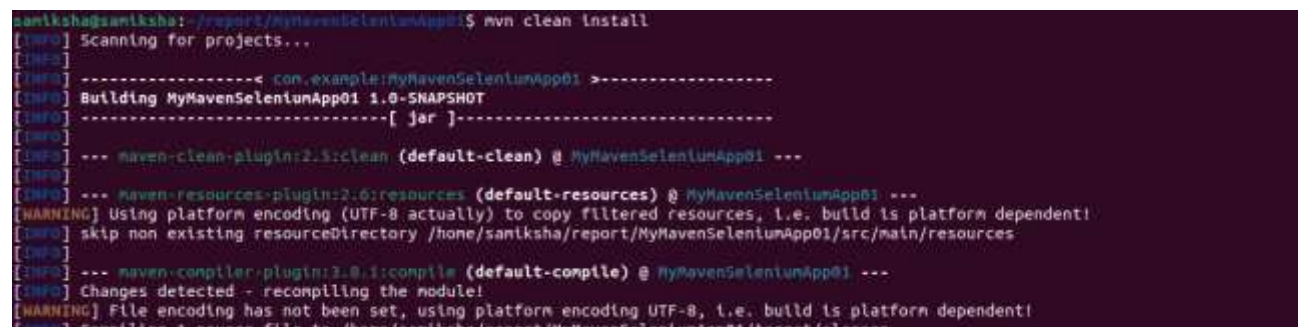


```
import org.openqa.selenium.chrome.ChromeDriver;

/**
 * Hello world!
 *
 */
public class App
{
    public static void main( String[] args )
    {
        WebDriver driver=new ChromeDriver();
        driver.get("https://www.saucedemo.com/");
        driver.manage().window().maximize();
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
    }
}
```

Step 5: build and run the maven application

Type **mvn clean install** in the terminal



```
saniksha@saniksha:~/report/MyMavenSeleniumApp01$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenSeleniumApp01 >-----
[INFO] Building MyMavenSeleniumApp01 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenSeleniumApp01 ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenSeleniumApp01 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/saniksha/report/MyMavenSeleniumApp01/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenSeleniumApp01 ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/saniksha/report/MyMavenSeleniumApp01/target/classes
```

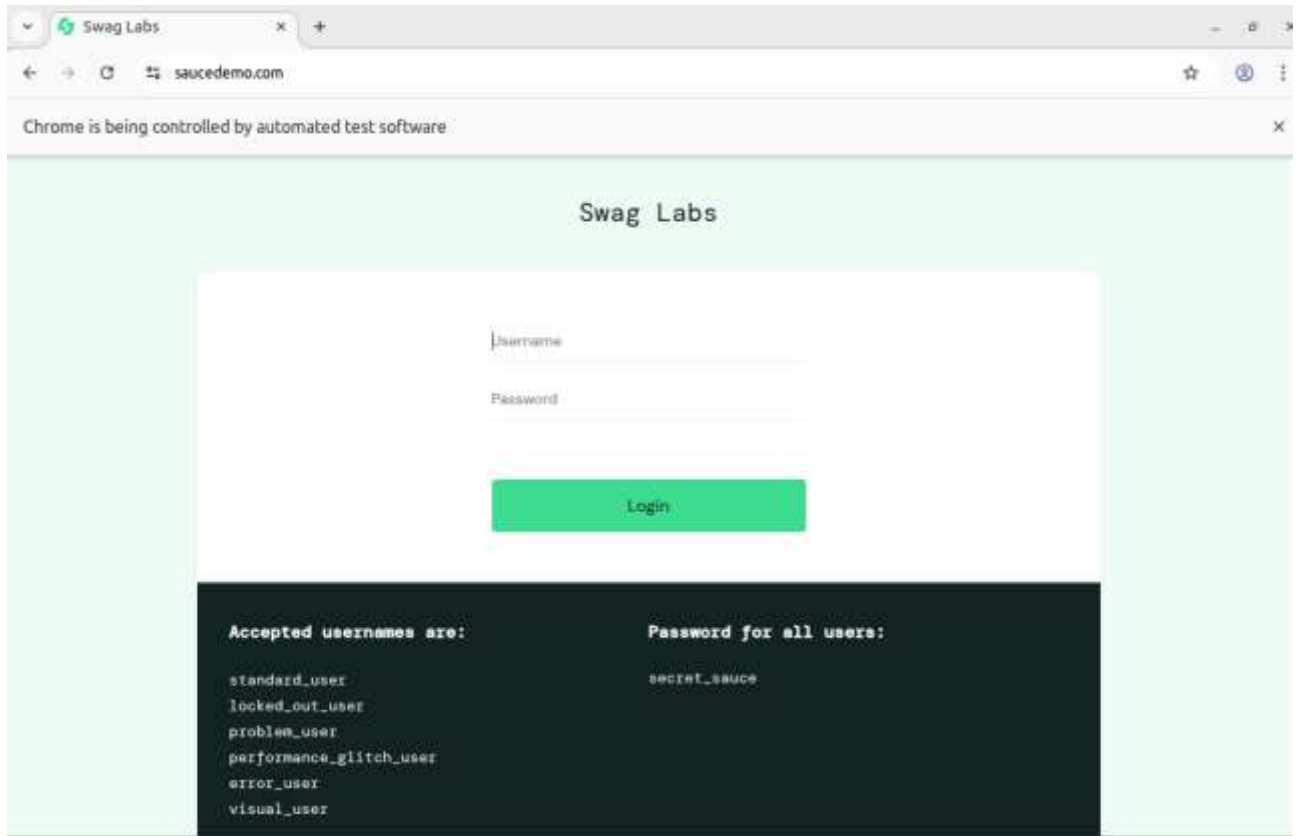
```

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.900 s
[INFO] Finished at: 2026-01-17T21:18:01+05:30
[INFO] -----

```

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**

it opens saucedemo.com and we can login using the credentials



Step 6: now open the terminal and type the command **cd MyMavenPracticeAutomationApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile pipeline {

agent any // Use any available agent

tools {

maven 'Maven' // Ensure this matches the name configured in Jenkins

}

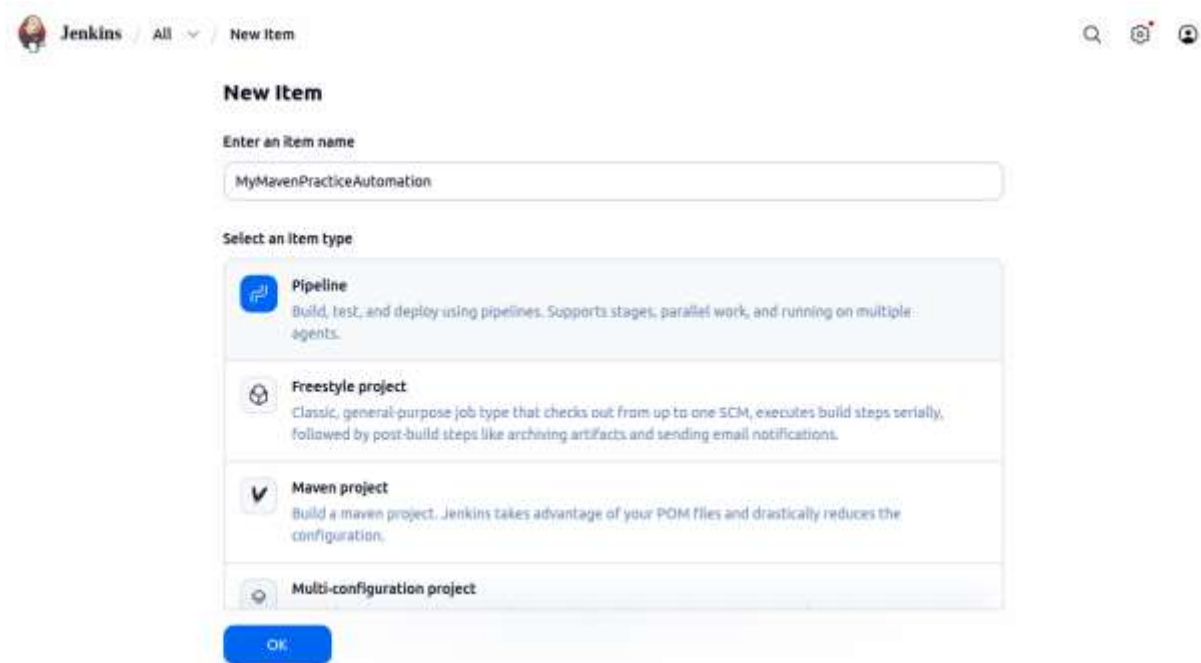
stages {

```
stage('Checkout') {
    steps {
        git
        branch: 'master',
        url: 'https://github.com/Samiksha201104/MyMavenPracticeAutomationApp.git'
    }
}
stage('Build') {
    steps {
        sh 'mvn clean package' // Run Maven build
    }
}
stage('Test') {
    steps {
        sh 'mvn test' // Run unit tests
    }
}
stage('Run Application') {
    steps {
        // Start the JAR application
        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
    }
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
```

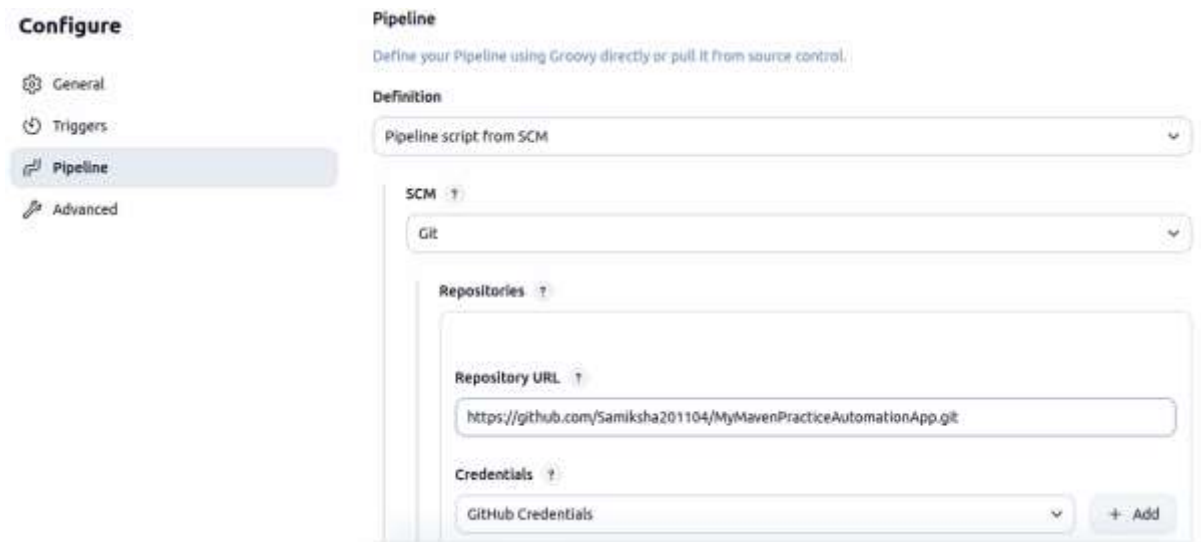
}}}

Then commit and push it to GitHub

Step 7: in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials



Step 8 : in the Dashboard click on the MyMavenPracticeAutomationApp item and select Build Now and then Full Stage view

DevOps(BCSL657D)

Add description

All +

S	W	Name ↓	Structure	Last Duration
✓	☁	MyGradleApp	1 min	5 min 51 sec
✓	☁	MyMavenApp	10 hr	33 sec
⋮	☀	MyMavenPracticeAutomationApp	N/A	N/A

- </> Changes
- ▶ Build Now
- ⚙ Configure
- 🗑 Delete Pipeline
- 🔍 Full Stage View
- 📁 Stages
- ✎ Rename
- ❓ Pipeline Syntax

MyMavenPracticeAutomationApp - Stage View



To correct the errors change
pom.xml

<source>1.7</source>

To

<target>1.7</target>

<source>17</source>

<target>17</target>

the code in the

Save and Build again in Jenkins



Working with Selenium 2

practiceautomation.com

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenPracticeAutomation -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**

```
sanikshasamiksha: /Devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenPracticeAutomation -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
INFO] -----
INFO] BUILD SUCCESS
INFO] -----
INFO] Total time: 4.456 s
INFO] Finished at: 2026-03-17T20:40:07+05:30
INFO] -----
```

Step 2 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenPracticeAutomationApp</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenPracticeAutomationApp</name>
  <url>http://maven.apache.org</url>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.29.0</version>
    </dependency>
  </dependencies>
```

```

    <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-jar-plugin</artifactId>
    <version>3.0.1</version>
    <configuration>
        <archive>
            <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
        </archive>
    </configuration>
</plugin>

</plugins>
</build>
</project>

```

Step 3: Edit the App.java

```

package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class App
{
    public static void main( String[] args )
    {
        WebDriver driver = new ChromeDriver();
        driver.get("https://practicetestautomation.com/practice-test-login/");
        driver.manage().window().maximize();
    }
}

```

```

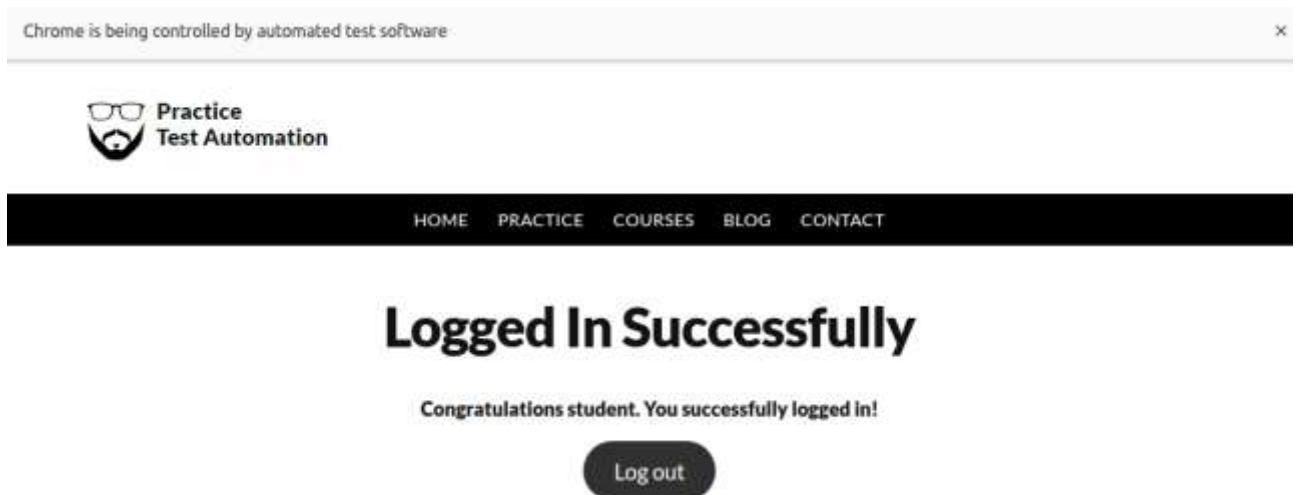
driver.findElement(By.id("username")).sendKeys("student");
driver.findElement(By.id("password")).sendKeys("Password123");
driver.findElement(By.id("submit")).click();
}
}

```

Step 4: build and run the maven application

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**

it opens practiceautomation.com and we can login using the credentials



Step 5: now open the terminal and type the command **cd MyMavenPracticeAutomationApp** to enter the folder and create a **Jenkinsfile**. Paste the code in the Jenkinsfile

```

pipeline {
    agent any // Use any available agent
    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
}

```



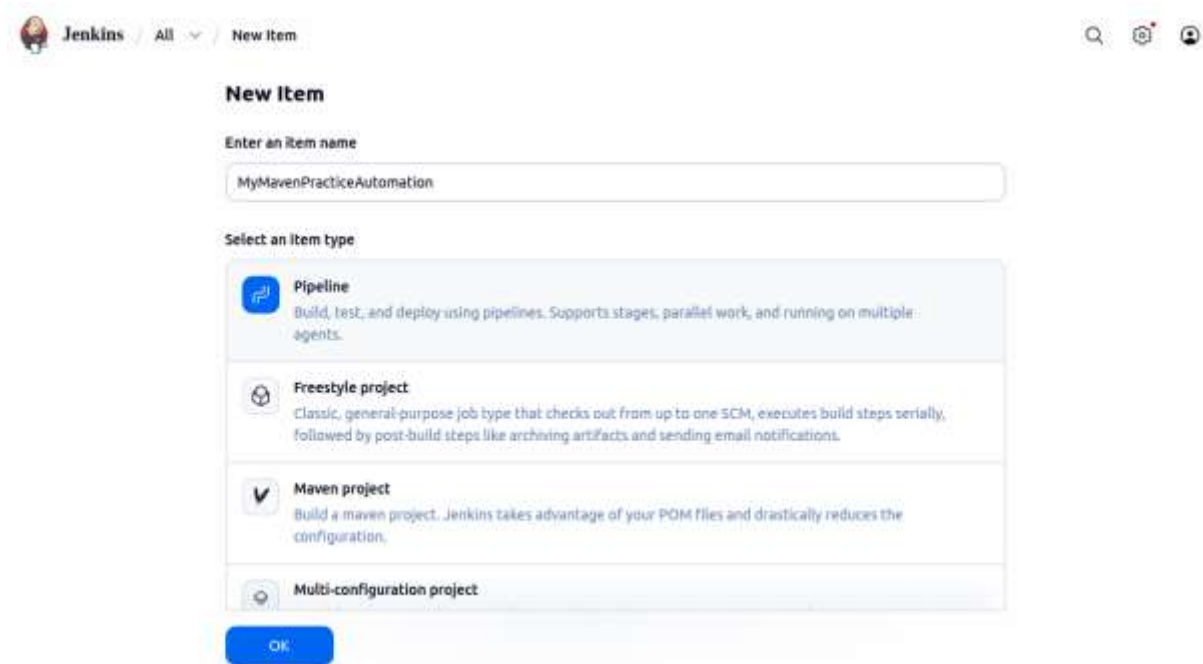
```
stages {
  stage('Checkout') {
    steps {
      git          branch:          'master',          url:
'https://github.com/Samiksha201104/MyMavenPracticeAutomationApp.git'
    }
  }
  stage('Build') {
    steps {
      sh 'mvn clean package' // Run Maven build
    }
  }
  stage('Test') {
    steps {
      sh 'mvn test' // Run unit tests
    }
  }
  stage('Run Application') {
    steps {
      // Start the JAR application
      sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
    }
  }
}
post {
  success {
    echo 'Build and deployment successful!'
  }
  failure {
```

```
echo 'Build failed!'
```

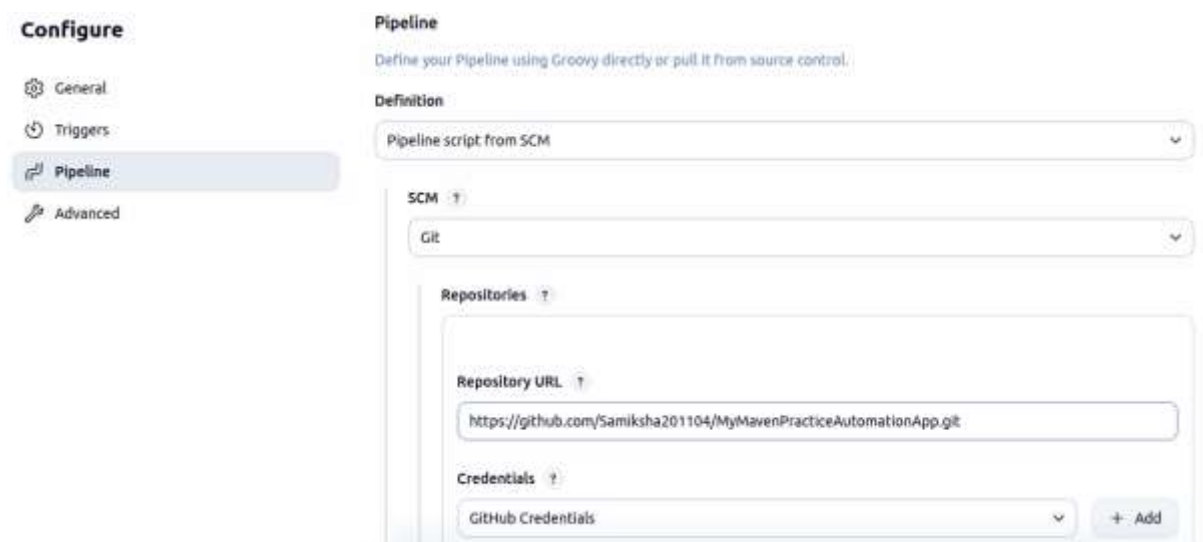
```
}}}
```

Then commit and push it to GitHub

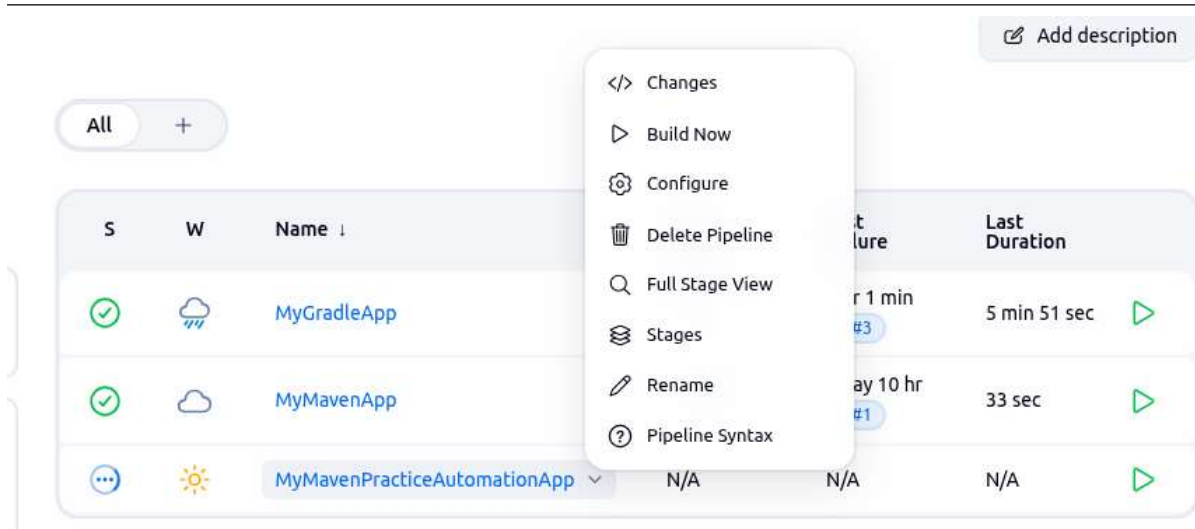
Step 6: in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials



Step 7 : in the Dashboard click on the MyMavenPracticeAutomationApp item and select Build Now and then Full Stage view



MyMavenPracticeAutomationApp - Stage View



To correct the errors change the code in the pom.xml

```
<source>1.7</source>
```

```
<target>1.7</target>
```

Save and Build again in Jenkins

Working with Selenium 3

automationexercise.com

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenAutomationExercise -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**

```
sanikshu@sanikshu: /Users/sanikshu/MyMavenPracticeAutomationApp$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenExerciseAutomationApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```

INFO] BUILD SUCCESS
INFO]
INFO] Total time: 4.456 s
INFO] Finished at: 2026-03-17T20:40:05:30
INFO]

```

Step 2 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

```

```

  <modelVersion>4.0.0</modelVersion>

```

```

  <groupId>com.example</groupId>

```

```

  <artifactId>MyMavenExerciseAutomationApp</artifactId>

```

```

  <packaging>jar</packaging>

```

```

  <version>1.0-SNAPSHOT</version>

```

```

  <name>MyMavenExerciseAutomationApp</name>

```

```

  <url>http://maven.apache.org</url>

```

```

  <dependencies>

```

```

    <dependency>

```

```

      <groupId>junit</groupId>

```

```

      <artifactId>junit</artifactId>

```

```

      <version>3.8.1</version>

```

```

      <scope>test</scope>

```

```

    </dependency>

```

```

  <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-
java -->

```

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.29.0</version>
</dependency>

<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.8.0</version>
</dependency>
</dependencies>

<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
```

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-jar-plugin</artifactId>
    <version>3.0.1</version>
    <configuration>
        <archive>
            <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
        </archive>
    </configuration>
</plugin>
</plugins>
</build>
</project>
```

Step 3: Edit the App.java

```
package com.example;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;

import io.github.bonigarcia.wdm.WebDriverManager;
import java.time.Duration;

public class App {
    public static void main(String[] args) {

        WebDriverManager.chromedriver().setup();
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();

        WebDriverWait wait = new WebDriverWait(driver,
        Duration.ofSeconds(10));

        try {
            driver.get("https://automationexercise.com");

            // Click Products page

            wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//a[@href='/products']"))).click();

            // Find first Add to Cart button
```

```
WebElement addToCart =
wait.until(ExpectedConditions.elementToBeClickable(
    By.xpath("//a[@data-product-id='1']")));

    // Scroll and click using JavaScript (avoids ad overlay)
    ((JavascriptExecutor)
driver).executeScript("arguments[0].scrollIntoView(true);", addToCart);
    ((JavascriptExecutor) driver).executeScript("arguments[0].click();",
addToCart);

    // Wait for confirmation modal

wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("cartModal"))
);

    // Click View Cart

wait.until(ExpectedConditions.elementToBeClickable(By.linkText("View
Cart"))).click();

    System.out.println("Product successfully added to cart!");

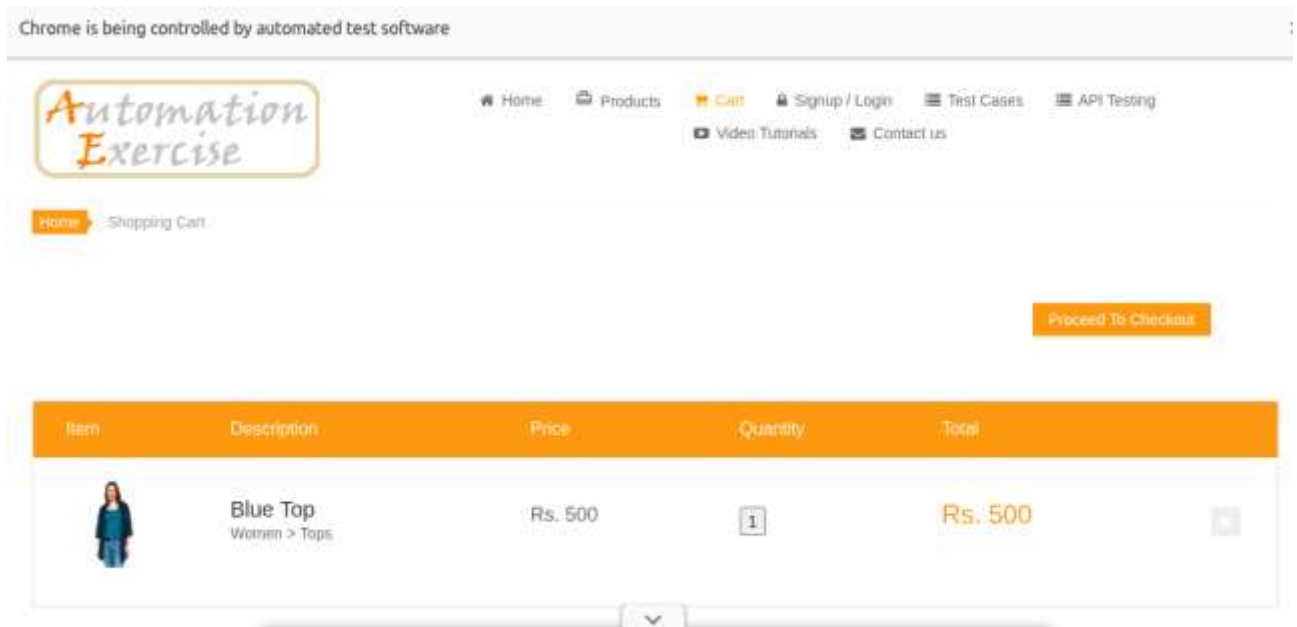
    } catch (Exception e) {
        System.out.println("TEST FAILED: " + e.getMessage());
    }

    System.out.println("Browser will remain open for inspection.");
}
}
```

Step 4: build and run the maven application

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**

it opens automationexercise.com and automatically adds a product to cart



Step 5: now open the terminal and type the command **cd MyMavenExerciseAutomationApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

pipeline {

agent any // Use any available agent

tools {

maven 'Maven' // Ensure this matches the name configured in Jenkins

}

stages {

stage('Checkout') {

steps {

git branch: 'master', url:

'https://github.com/Samiksha201104/MyMavenExerciseAutomationApp.git'

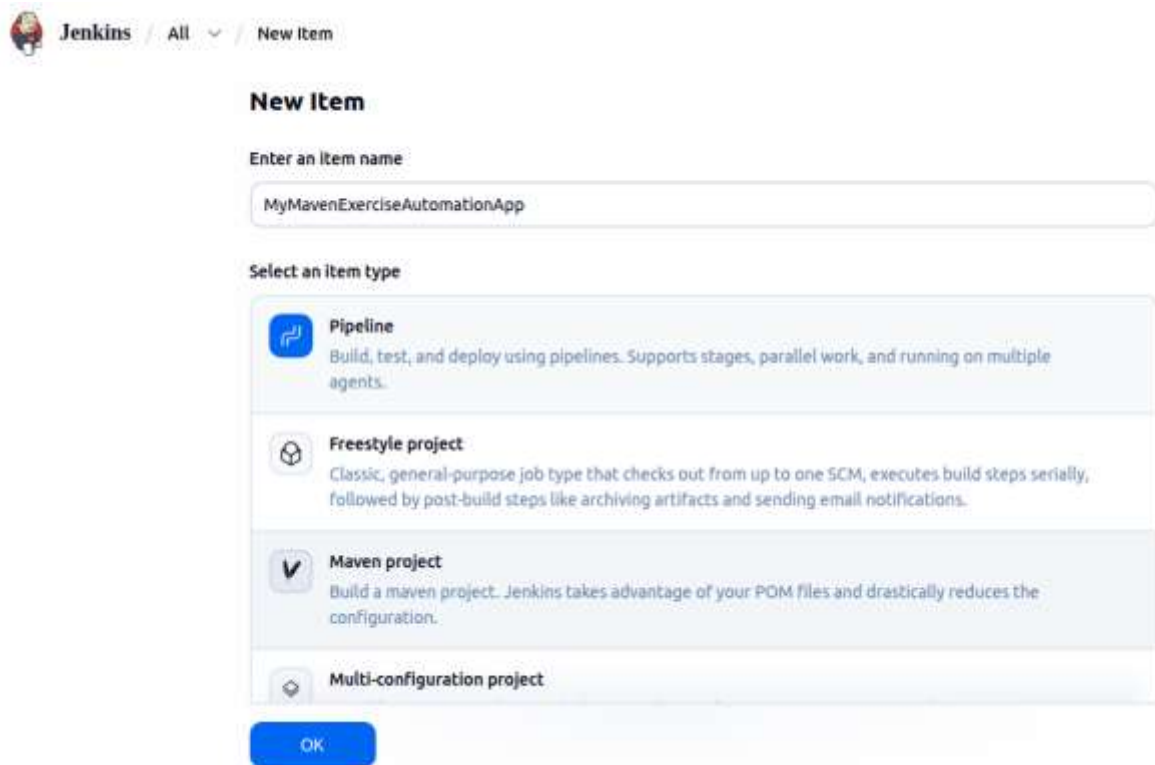
}

}

```
stage('Build') {
    steps {
        sh 'mvn clean package' // Run Maven build
    }
}
stage('Test') {
    steps {
        sh 'mvn test' // Run unit tests
    }
}
stage('Run Application') {
    steps {
        // Start the JAR application
        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
    }
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}}
```

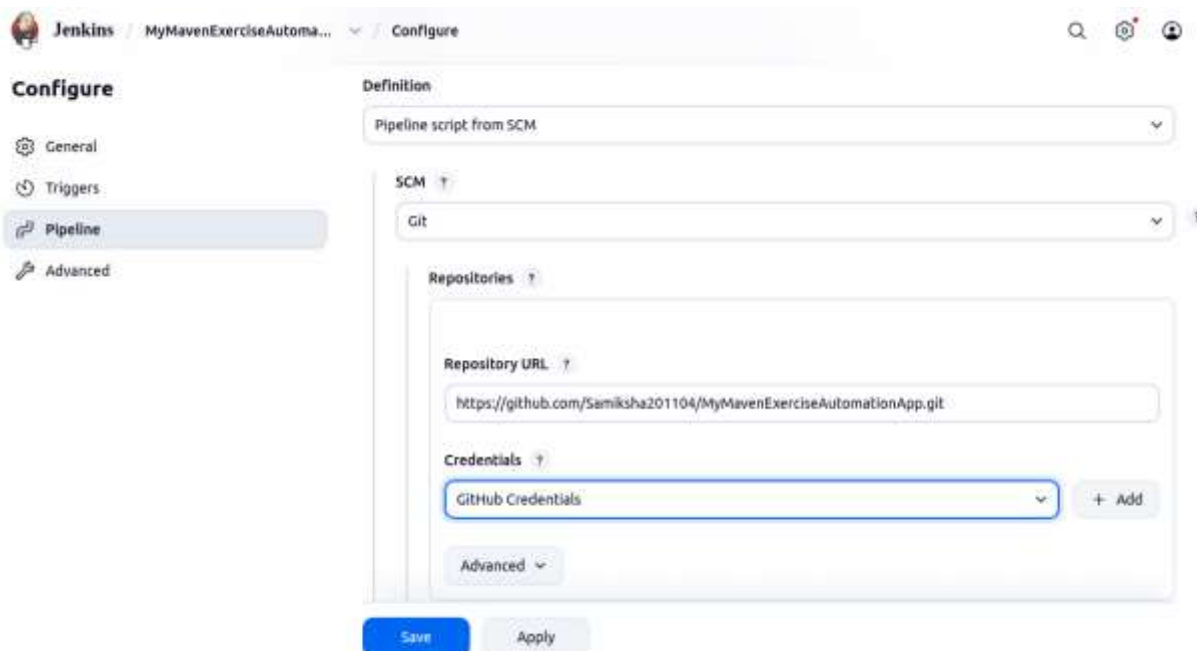
Then commit and push it to GitHub

Step 6:in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



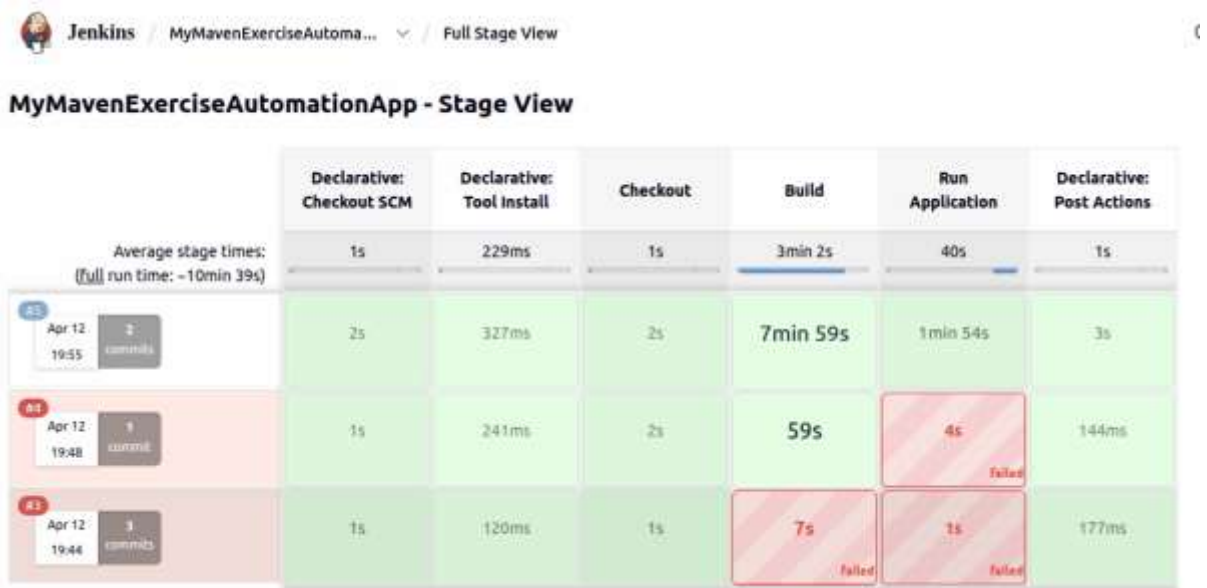
The image shows the Jenkins 'New Item' dialog. At the top, there's a breadcrumb 'Jenkins / All / New Item'. The title is 'New Item'. Below it, a text field 'Enter an Item name' contains 'MyMavenExerciseAutomationApp'. Under 'Select an item type', there are four options: 'Pipeline' (with a blue icon), 'Freestyle project' (with a cube icon), 'Maven project' (with a checkmark icon), and 'Multi-configuration project' (with a diamond icon). The 'Maven project' option is selected. At the bottom is an 'OK' button.

In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credentials



The image shows the Jenkins 'Configure' page for the item 'MyMavenExerciseAutomationApp'. The left sidebar has 'Configure' at the top, followed by 'General', 'Triggers', 'Pipeline' (which is selected and highlighted), and 'Advanced'. The main area is titled 'Definition' and shows 'Pipeline script from SCM' selected in a dropdown. Below this, the 'SCM' dropdown is set to 'Git'. Under 'Repositories', there's a 'Repository URL' field containing 'https://github.com/Samiksha201104/MyMavenExerciseAutomationApp.git'. Below that, the 'Credentials' dropdown is set to 'GitHub Credentials', with an '+ Add' button to the right. At the bottom are 'Save' and 'Apply' buttons.

Step 7 : in the Dashboard click on the MyMavenExerciseAutomationApp item and select Build Now and then Full Stage view



Program 6:

Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation

Step 1: create directory using command **mkdir MyGradleApp** and enter the directory using the command **cd MyGradleApp**

```
samiksha@samiksha:~/Devops6D$ mkdir MyGradleapp
samiksha@samiksha:~/Devops6D$ cd MyGradleapp
samiksha@samiksha:~/Devops6D/MyGradleapp$
```

Step 2: Type the command **gradle init --type java-application**

```
samiksha@samiksha:~/Devops6D/MyGradleapp$ gradle init --type java-application
Enter target Java version (min: 7, default: 21): 21
Project name (default: MyGradleapp): MyGradleapp
Select application structure:
  1: Single application project
  2: Application and library project
Enter selection (default: Single application project) [1..2] 1
Select build script DSL:
  1: Kotlin
  2: Groovy
Enter selection (default: Kotlin) [1..2] 2
Select test framework:
  1: JUnit 4
  2: TestNG
  3: Spock
  4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 1
Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] no

> Task :init
To learn more about Gradle by exploring our Samples at https://docs.gradle.org/8.7/samples/sample_building_java_applications.html

BUILD SUCCESSFUL in 2m 9s
1 actionable task: 1 executed
<-----> 0% WAITING
> IDLE
samiksha@samiksha:~/Devops6D/MyGradleapp$
```

Step 3: Go to app directory to find build.gradle. Open build.gradle and type the code in build.gradle

```
task hello {
    doLast {
        println '6D Devops Class'
    }
}
```

Step 4: type the command **gradle build** to compile the source code and package the app.

```
samiksha@samiksha:~/Devops6D/MyGradleapp$ gradle build
Starting a Gradle Daemon, 1 incompatible and 1 stopped Daemons could not be reused, use --status for details
Path for java installation '/usr/lib/jvm/openjdk-11' (Common Linux Locations) does not contain a java executable
Path for java installation '/usr/lib/jvm/openjdk-17' (Common Linux Locations) does not contain a java executable

BUILD SUCCESSFUL in 1m 5s
7 actionable tasks: 7 executed
-----
```

Step 5: type the command **gradle run** to execute the default Hello World application

```

samiksha@samiksha:~/Devops6D/MyGradleapp$ gradle run
Path for java installation '/usr/lib/jvm/openjdk-17' (Common Linux Locations) does not contain a java executable
Path for java installation '/usr/lib/jvm/openjdk-11' (Common Linux Locations) does not contain a java executable

> Task :app:run
Hello World!

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date

```

Step 6: type the command **gradle <task-name>** to run the task

```

samiksha@samiksha:~/Devops6D/MyGradleapp$ gradle hello

> Task :app:hello
6D Devops Class

BUILD SUCCESSFUL in 1s
1 actionable task: 1 executed

```

Step 7: in the terminal type the code MyGradleApp to enter the folder .

Type the command `gedit JenkinsFile` to open the Jenkinsfile and paste the code pipeline {

```

    agent any // Use any available agent

    tools {
        gradle 'Gradle' // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/Samiksha201104/MyGradleApp.git'
            }
        }
        stage('Build') {
            steps {

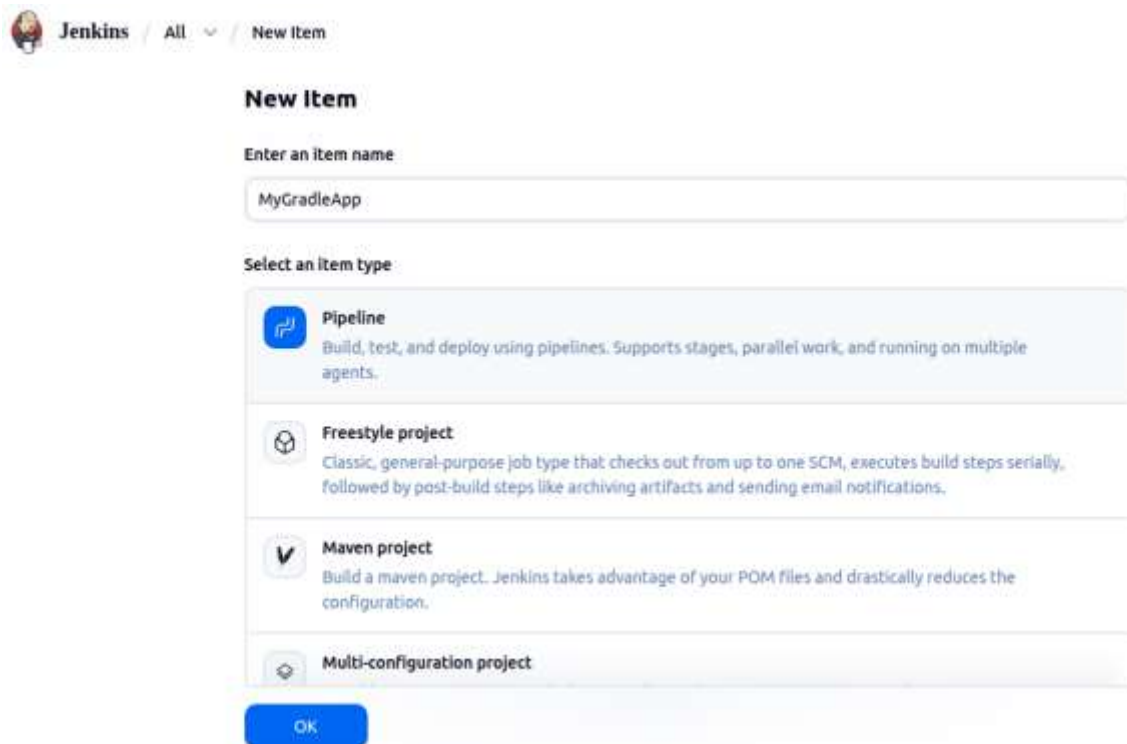
```

```
        sh 'gradle build' // Run Gradle build
    }
}
stage('Test') {
    steps {
        sh 'gradle test' // Run unit tests
    }
}
stage('Run Application') {
    steps {
        // Start the JAR application
        sh 'gradle run'
    }
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
```

Commit and push to GitHub

Step 8: in Jenkins Dashboard click on New Item

Give the name MyGradleApp and select Pipeline



The image shows the 'New Item' dialog in Jenkins. At the top, there's a breadcrumb 'Jenkins / All / New Item'. The title is 'New Item'. Below it, a text field 'Enter an item name' contains 'MyGradleApp'. Under 'Select an item type', there are four options: 'Pipeline' (selected), 'Freestyle project', 'Maven project', and 'Multi-configuration project'. Each option has a brief description. At the bottom is an 'OK' button.

New Item

Enter an item name

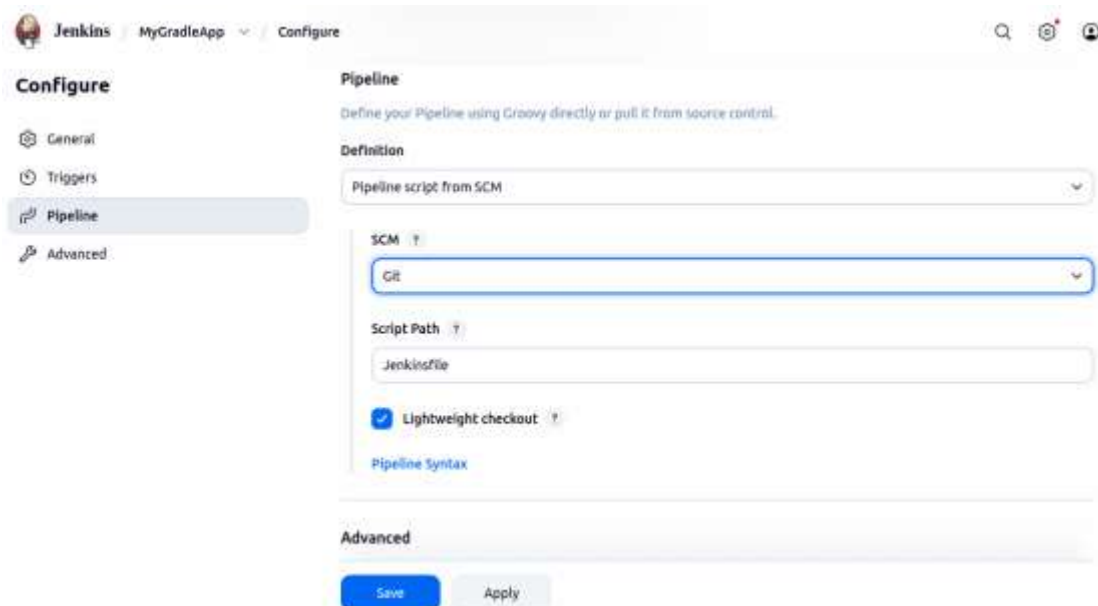
MyGradleApp

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**

OK

In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credentials



The image shows the 'Configure' page for the 'MyGradleApp' job in Jenkins. The left sidebar has 'Configure' at the top, followed by 'General', 'Triggers', 'Pipeline' (selected), and 'Advanced'. The main content area is titled 'Pipeline' and has a subtitle 'Define your Pipeline using Groovy directly or pull it from source control.' Under 'Definition', there's a dropdown menu set to 'Pipeline script from SCM'. Below that, 'SCM' is set to 'Git'. 'Script Path' is set to 'Jenkinsfile'. The 'Lightweight checkout' checkbox is checked. There's a link for 'Pipeline Syntax'. At the bottom, there's an 'Advanced' section with 'Save' and 'Apply' buttons.

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM

Git

Script Path

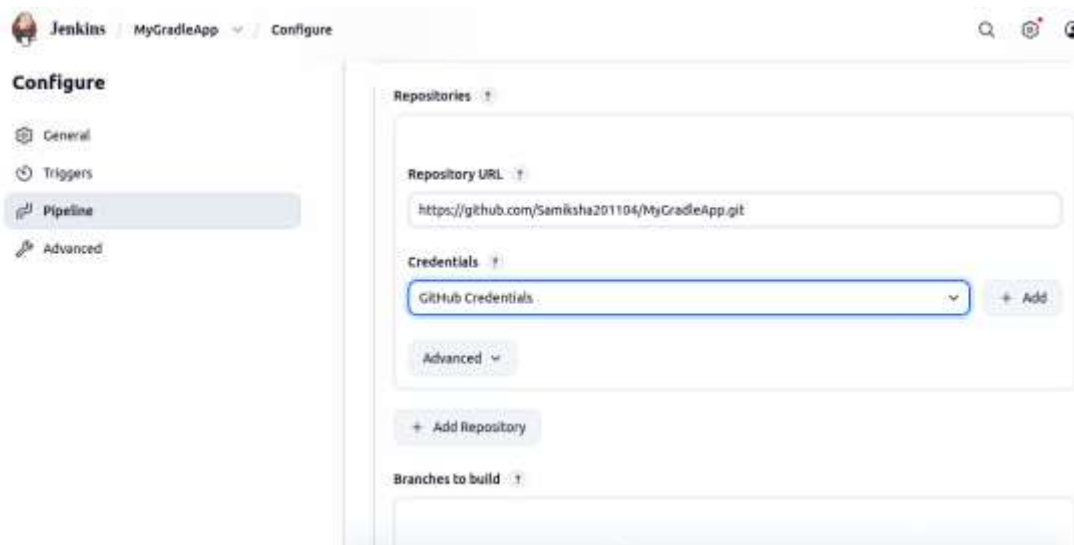
Jenkinsfile

☒ Lightweight checkout

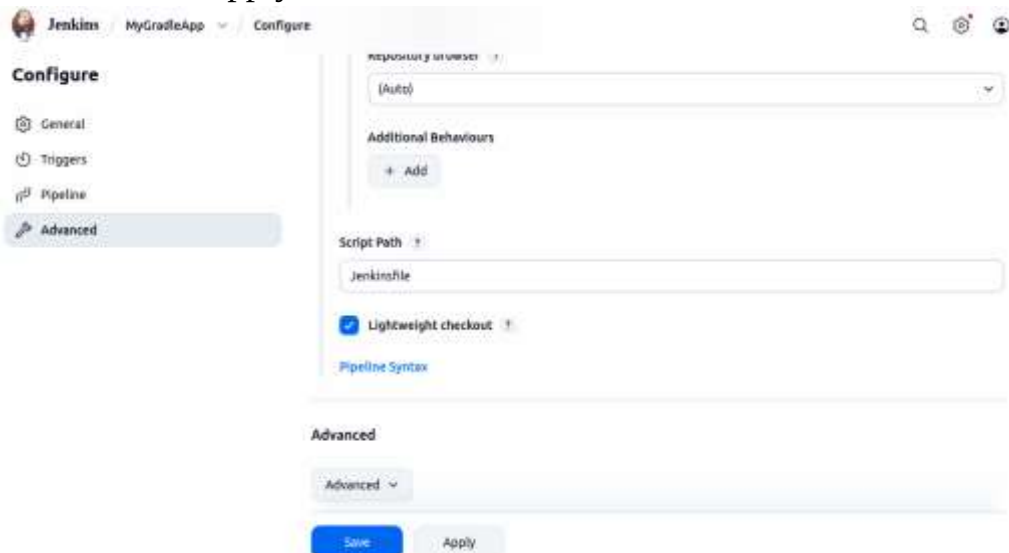
[Pipeline Syntax](#)

Advanced

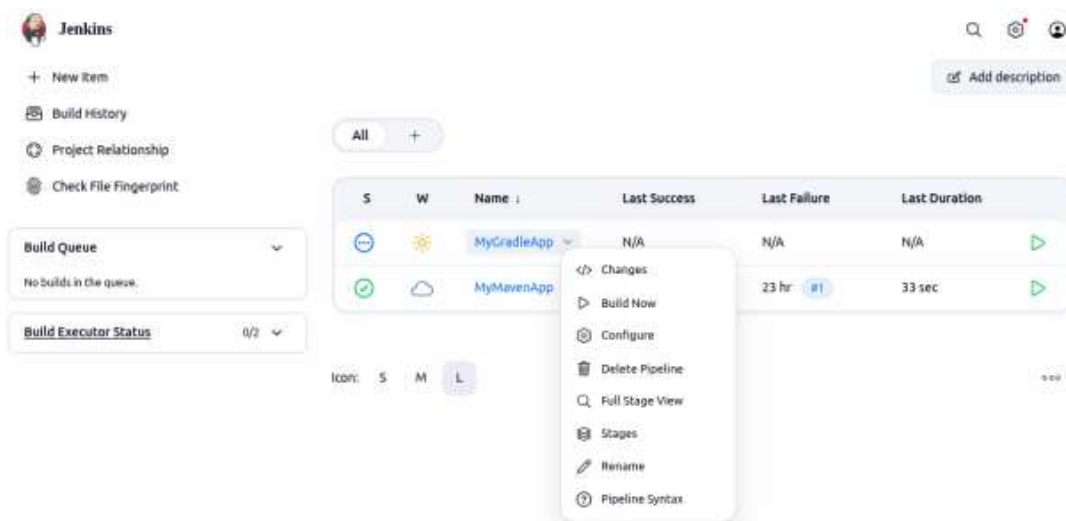
Save Apply



Then click on apply and save



Step 9: go to Jenkins Dashboard select the MyGradleApp pipeline and select Build Now. After build select Full Stage View



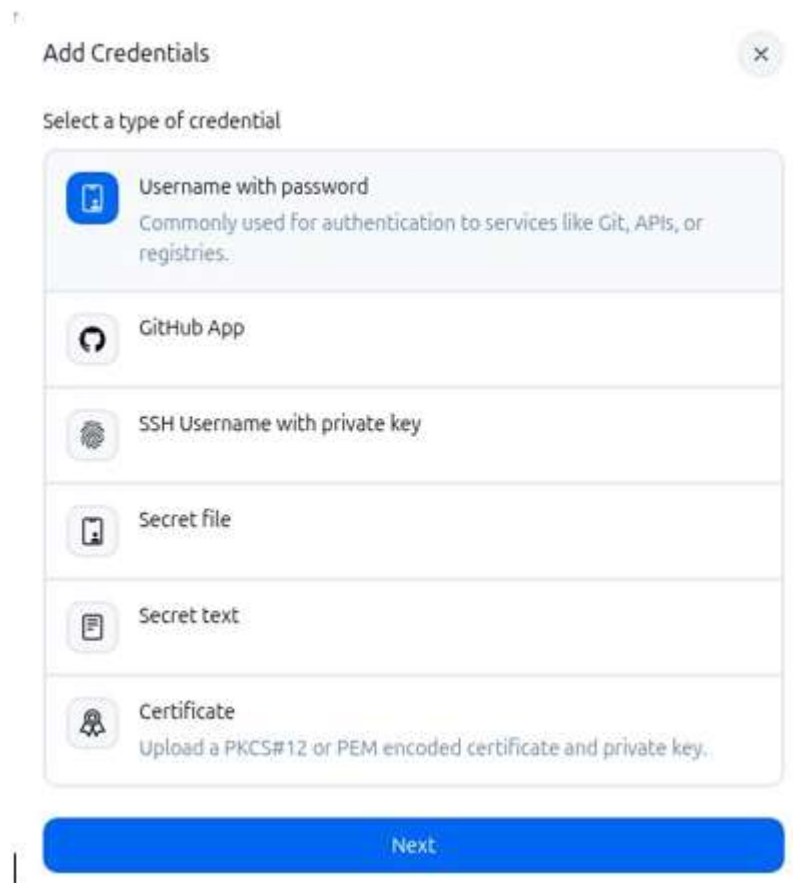
Once the errors are corrected Build again



Continuous Integration (CI) Pipeline with Jenkins & Maven/Gradle

Step 1: Login to Jenkins and go to the Credentials page and click on add credentials

Step 2: Add new credentials in Jenkins for CI pipeline. Enter details like username, password, set the scope to Global, and provide a description.



Step 3: now open the terminal and type the command **cd MyMavenApp** to enter the folder and create a **Jenkinsfile**. Paste the code in the Jenkinsfile pipeline {

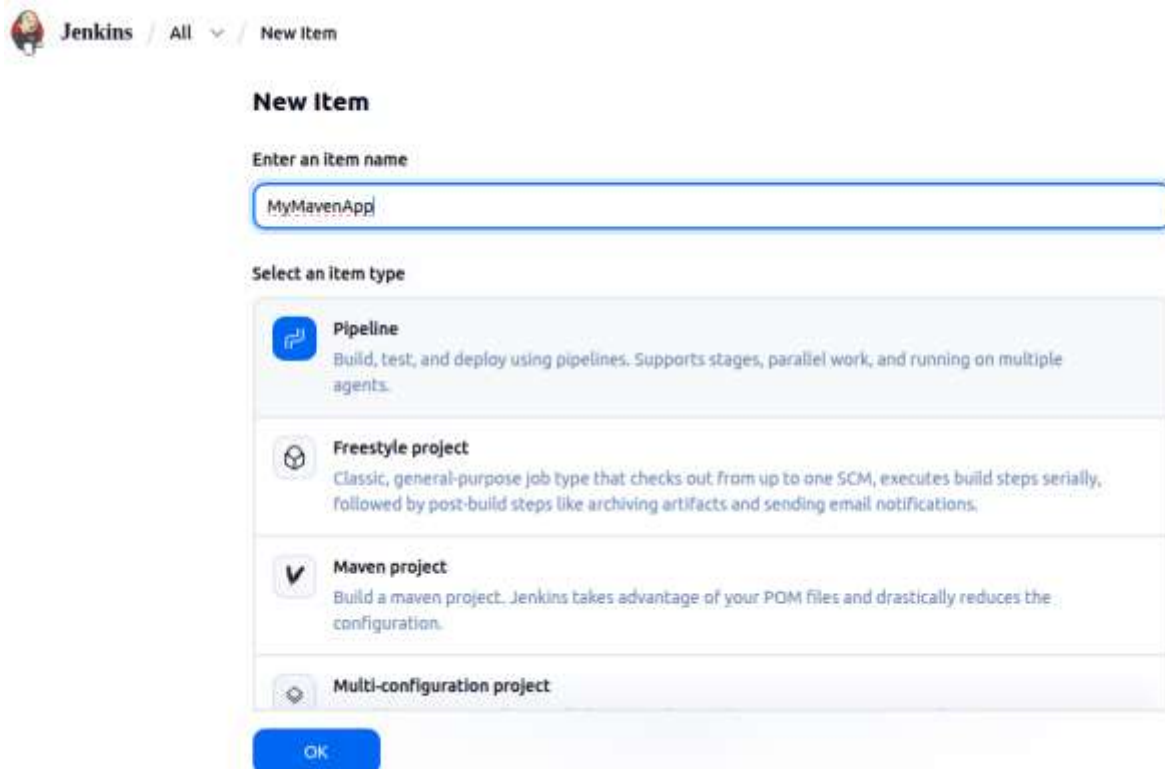
```
agent any // Use any available agent
tools {
    maven 'Maven' // Ensure this matches the name configured in Jenkins
}
stages {
    stage('Checkout') {
```

```
    steps {
      git          branch:          'master',          url:
'https://github.com/Samiksha201104/MyMavenApp.git'
    }
  }
  stage('Build') {
    steps {
      sh 'mvn clean package' // Run Maven build
    }
  }
  stage('Test') {
    steps {
      sh 'mvn test' // Run unit tests
    }
  }
  stage('Run Application') {
    steps {
      // Start the JAR application
      sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
    }
  }
}
post {
  success {
    echo 'Build and deployment successful!'
  }
  failure {
    echo 'Build failed!'
  }
}
```

Then commit and push it to GitHub

```
samiksha@samiksha:~/Devops6D/MyMavenApp$ git init
Reinitialized existing Git repository in /home/samiksha/Devops6D/MyMavenApp/.git/
samiksha@samiksha:~/Devops6D/MyMavenApp$ git add .
samiksha@samiksha:~/Devops6D/MyMavenApp$ git commit -m "Adding Jenkinsfile"
[master 8735bc6] Adding Jenkinsfile
 1 file changed, 47 insertions(+)
 create mode 100644 Jenkinsfile
samiksha@samiksha:~/Devops6D/MyMavenApp$ git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 729 bytes | 729.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Samiksha201104/Devops6D.git
 c5925f2..8735bc6  master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
samiksha@samiksha:~/Devops6D/MyMavenApp$ █
```

Step 4: Go to Jenkins Dashboard and click on new item , enter the name as MyMavenApp and choose pipeline



Jenkins / All ▾ / New Item

New Item

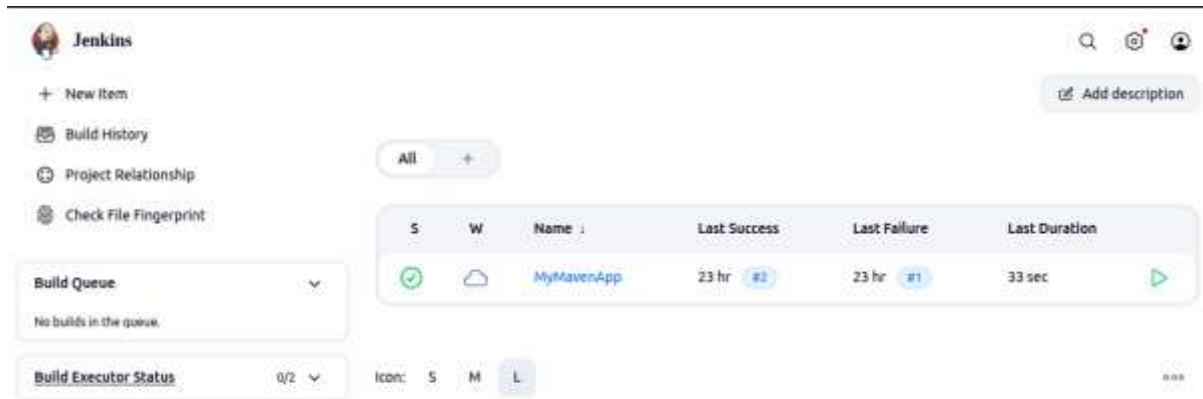
Enter an item name

Select an item type

-  **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
-  **Multi-configuration project**

OK

Step 5: in the Jenkins dashboard select MyMavenApp and select build now and then select full stage view



Step 6 : Note the stages that failed and check the error by clicking the stage and selecting logs.

MyMavenApp - Stage View



To correct the error open pom.xml and make changes in the code

The original code has the following lines

```
<source>1.7</source>
```

```
<target>1.7</target>
```

Change this code to

```
<source>11</source>
```

```
<target>11</target>
```

Push this back to GitHub and build it again on Jenkins

Gradle with Jenkins

Step 1: in the terminal type the code MyGradleApp to enter the folder .

Type the command gedit JenkinsFile to open the Jenkinsfile and paste the code pipeline {

```
    agent any // Use any available agent

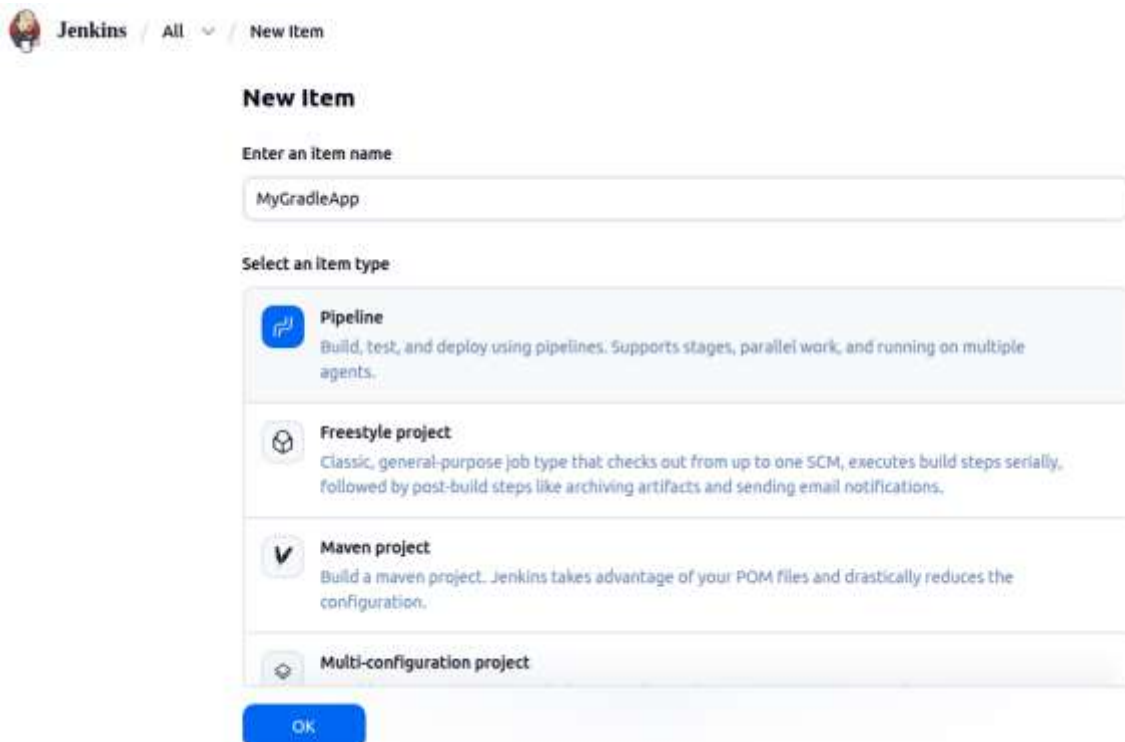
    tools {
        gradle 'Gradle' // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/Samiksha201104/MyGradleApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'gradle build' // Run Gradle build
            }
        }
        stage('Test') {
            steps {
                sh 'gradle test' // Run unit tests
            }
        }
        stage('Run Application') {
```

```
steps {  
    // Start the JAR application  
    sh 'gradle run'  
}  
}  
post {  
    success {  
        echo 'Build and deployment successful!'  
    }  
    failure {  
        echo 'Build failed!'  
    }  
}
```

Commit and push to GitHub

Step 2: in Jenkins Dashboard click on New Item

Give the name MyGradleApp and select Pipeline



Jenkins / All / New Item

New Item

Enter an item name

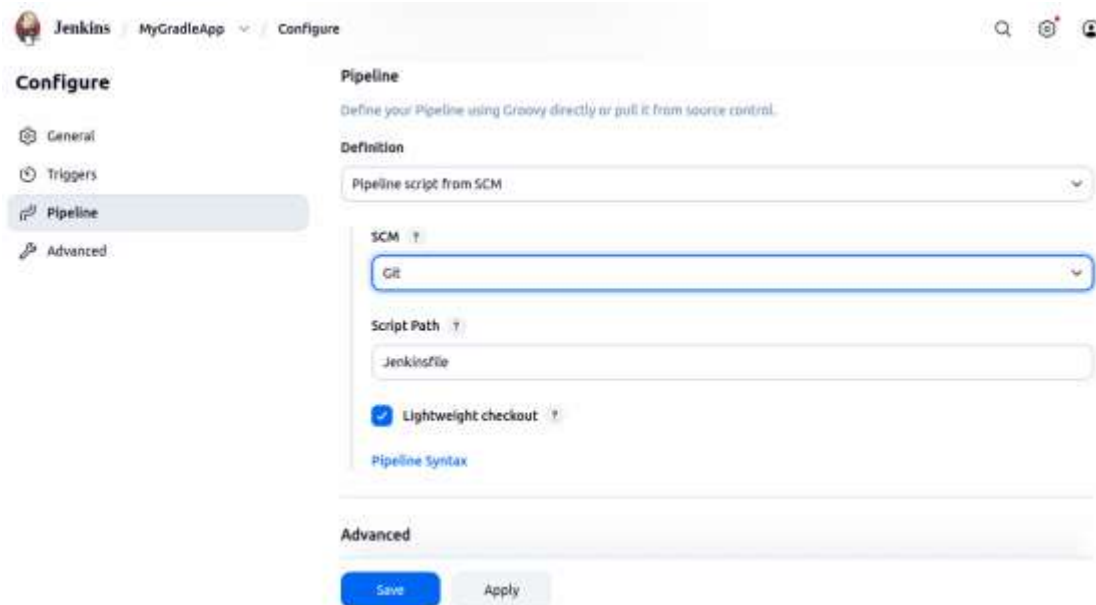
MyGradleApp

Select an Item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**

OK

In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials



This screenshot shows the Jenkins configuration page for a pipeline named 'MyGradleApp'. The 'Pipeline' tab is selected in the left sidebar. The 'Definition' section is expanded, showing 'Pipeline script from SCM' as the definition type. The 'SCM' dropdown is set to 'Git'. The 'Script Path' is set to 'Jenkinsfile'. The 'Lightweight checkout' checkbox is checked. At the bottom, there are 'Save' and 'Apply' buttons.

Jenkins / MyGradleApp / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM

Git

Script Path

Jenkinsfile

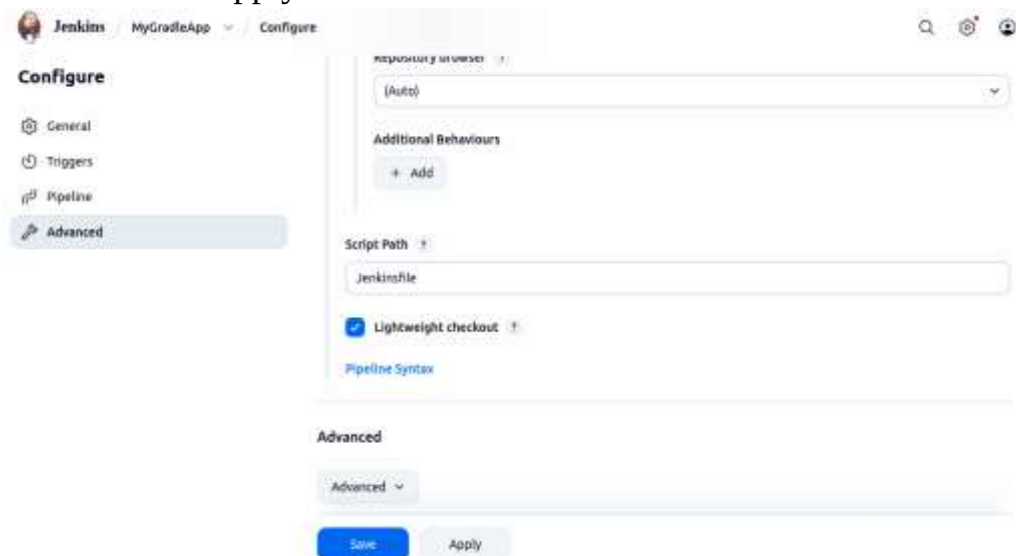
☒ Lightweight checkout

[Pipeline Syntax](#)

Advanced

Save Apply

Then click on apply and save



This screenshot shows the 'Advanced' configuration section of the Jenkins pipeline. The 'Repository URL' is set to 'https://github.com/Samiksha201104/MyGradleApp.git'. The 'Credentials' dropdown is set to 'GitHub Credentials'. The 'Add Repository' button is visible. The 'Script Path' is set to 'Jenkinsfile'. The 'Lightweight checkout' checkbox is checked. At the bottom, there are 'Save' and 'Apply' buttons.

Jenkins / MyGradleApp / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Repositories

Repository URL

https://github.com/Samiksha201104/MyGradleApp.git

Credentials

GitHub Credentials

+ Add

Advanced

+ Add Repository

Branches to build

Script Path

Jenkinsfile

☒ Lightweight checkout

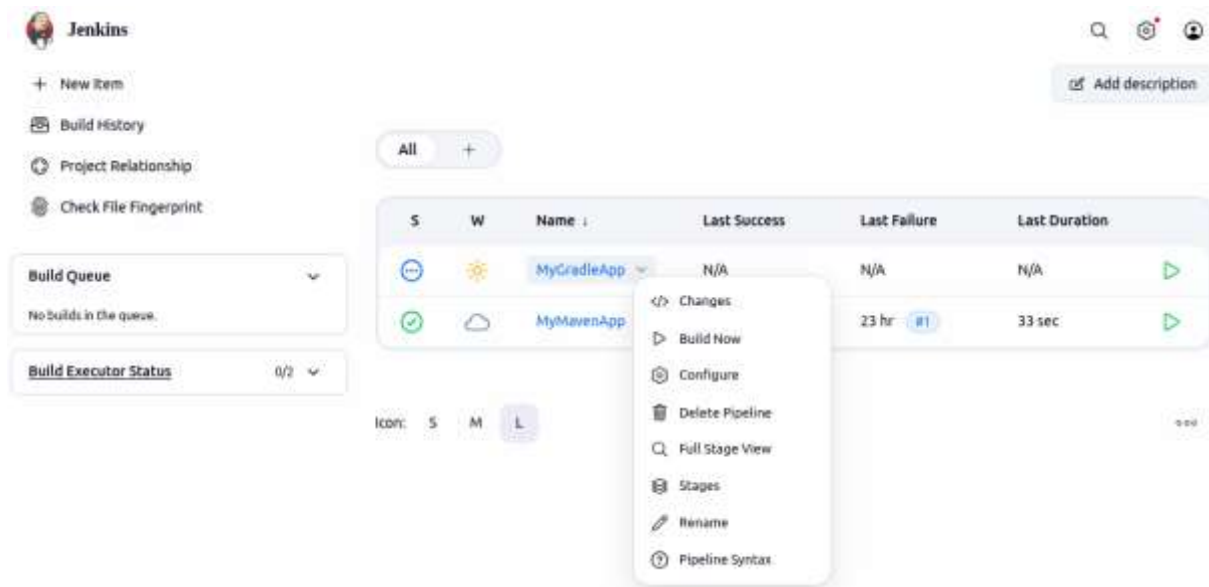
[Pipeline Syntax](#)

Advanced

Advanced

Save Apply

Step 3: go to Jenkins Dashboard select the MyGradleApp pipeline and select Build Now. After build select Full Stage View



Once the errors are corrected Build again

Jenkins / MyGradleApp / Full Stage View

MyGradleApp - Stage View

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times:	3s	536ms	2s	2min 50s	51s	15s	7s
Apr 10 21:49 1 commit	3s	314ms	2s	1min 40s	1min 14s	50s	13s
Apr 10 21:29 2 commits	2s	301ms	2s	4min 48s	1min 36s failed	8s failed	8s
Apr 10 21:10 2 commits	4s	743ms	1s	4min 43s failed	32s failed	2s failed	6s
Apr 10 11:28 No Changes	2s	788ms	2s	10s failed	654ms failed	204ms failed	322ms

MyMavenPracticeAutomtion

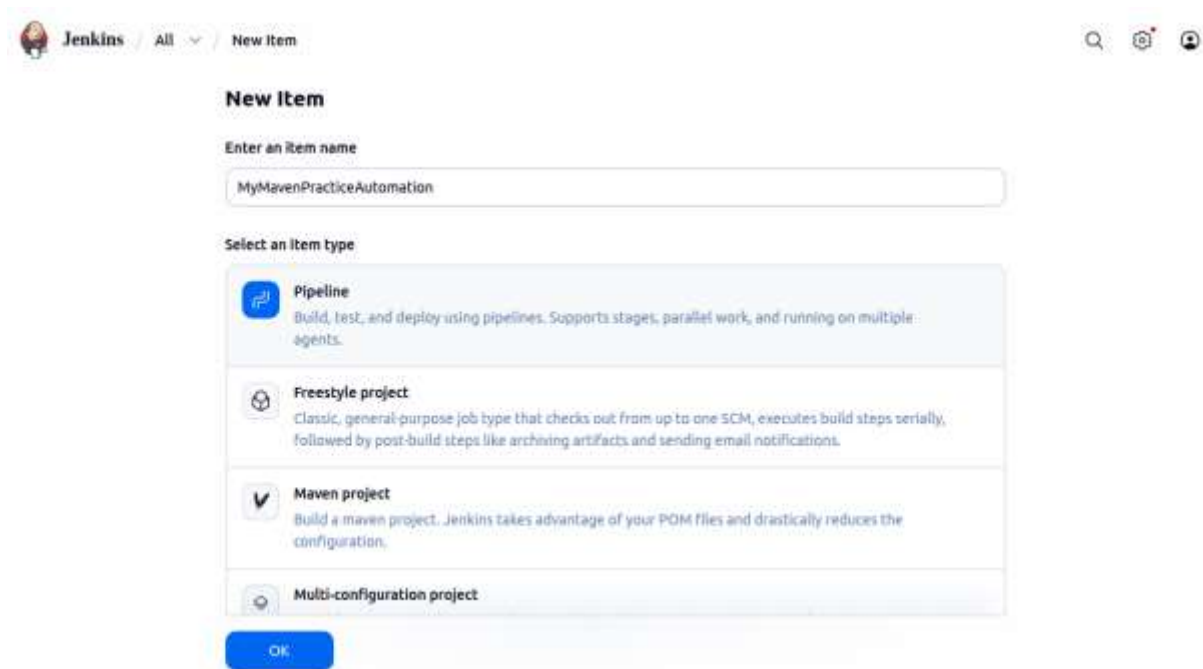
Step 1: now open the terminal and type the command **cd MyMavenPracticeAutomationApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

```
pipeline {
    agent any // Use any available agent
    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/Samiksha201104/MyMavenPracticeAutomationApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package' // Run Maven build
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test' // Run unit tests
            }
        }
        stage('Run Application') {
            steps {
```

```
// Start the JAR application
sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
}
}
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}}
```

Then commit and push it to GitHub

Step 2: in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

<https://github.com/Samiksha201104/MyMavenPracticeAutomationApp.git>

Credentials

GitHub Credentials

+ Add

Step 3 : in the Dashboard click on the MyMavenPracticeAutomationApp item and select Build Now and then Full Stage view

Add description

All +

S	W	Name ↓	Structure	Last Duration
✓	☁	MyGradleApp	1 min	5 min 51 sec
✓	☁	MyMavenApp	10 hr	33 sec
⋮	☀	MyMavenPracticeAutomationApp	N/A	N/A

- </> Changes
- ▶ Build Now
- ⚙ Configure
- 🗑 Delete Pipeline
- 🔍 Full Stage View
- 📁 Stages
- ✎ Rename
- ❓ Pipeline Syntax

MyMavenPracticeAutomationApp - Stage View



To correct the errors change
pom.xml

<source>1.7</source>

To

<source>17</source>

<target>17</target>

the code in the

<target>1.7</target>

Save and Build again in Jenkins



MyMavenExerciseAutomationApp

Step 1: now open the terminal and type the command **cd MyMavenExerciseAutomationApp** to enter the folder and create a **Jenkinsfile**. Paste the code in the Jenkinsfile pipeline {

```
agent any // Use any available agent
```

```
tools {
```

```
    maven 'Maven' // Ensure this matches the name configured in Jenkins
```

```
}
```

```
stages {
```

```
    stage('Checkout') {
```

```
        steps {
```

```
            git
```

```
                branch:
```

```
                'master',
```

```
                url:
```

```
'https://github.com/Samiksha201104/MyMavenExerciseAutomationApp.git'
```

```
        }
```

```
    }
```

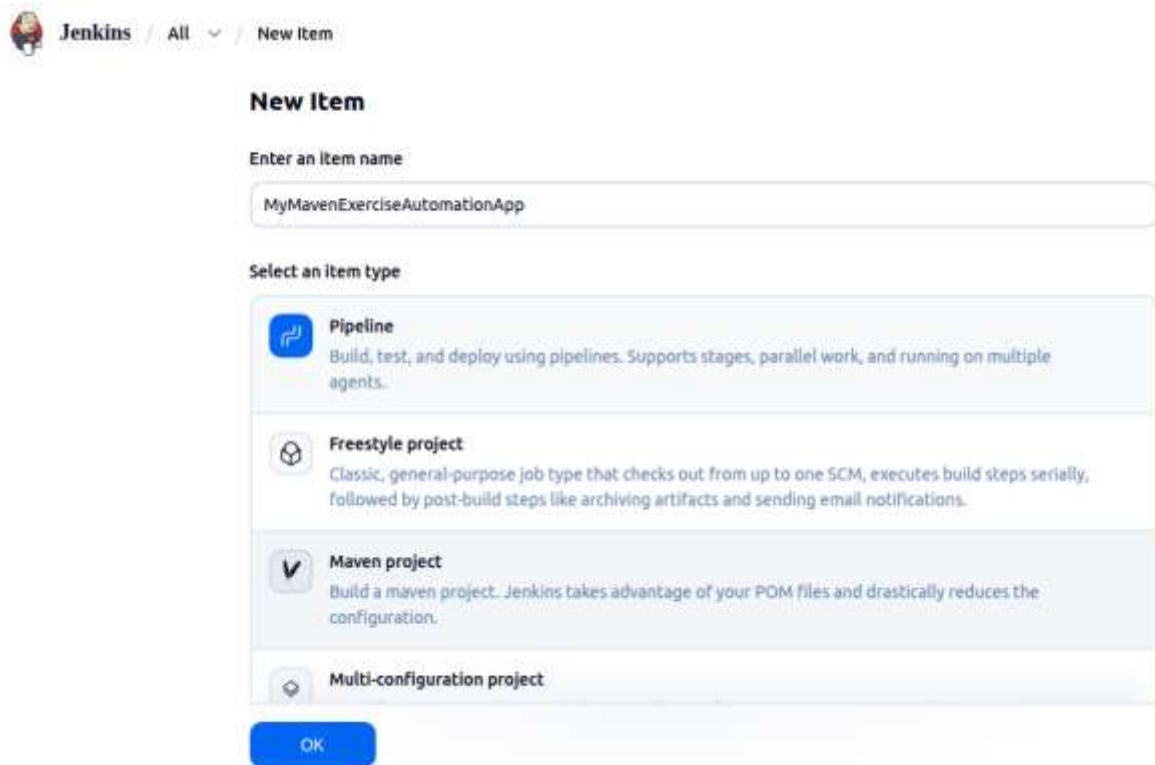
```
    stage('Build') {
```

```
        steps {
```

```
    sh 'mvn clean package' // Run Maven build
  }
}
stage('Test') {
  steps {
    sh 'mvn test' // Run unit tests
  }
}
stage('Run Application') {
  steps {
    // Start the JAR application
    sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
  }
}
}
post {
  success {
    echo 'Build and deployment successful!'
  }
  failure {
    echo 'Build failed!'
  }
}}
```

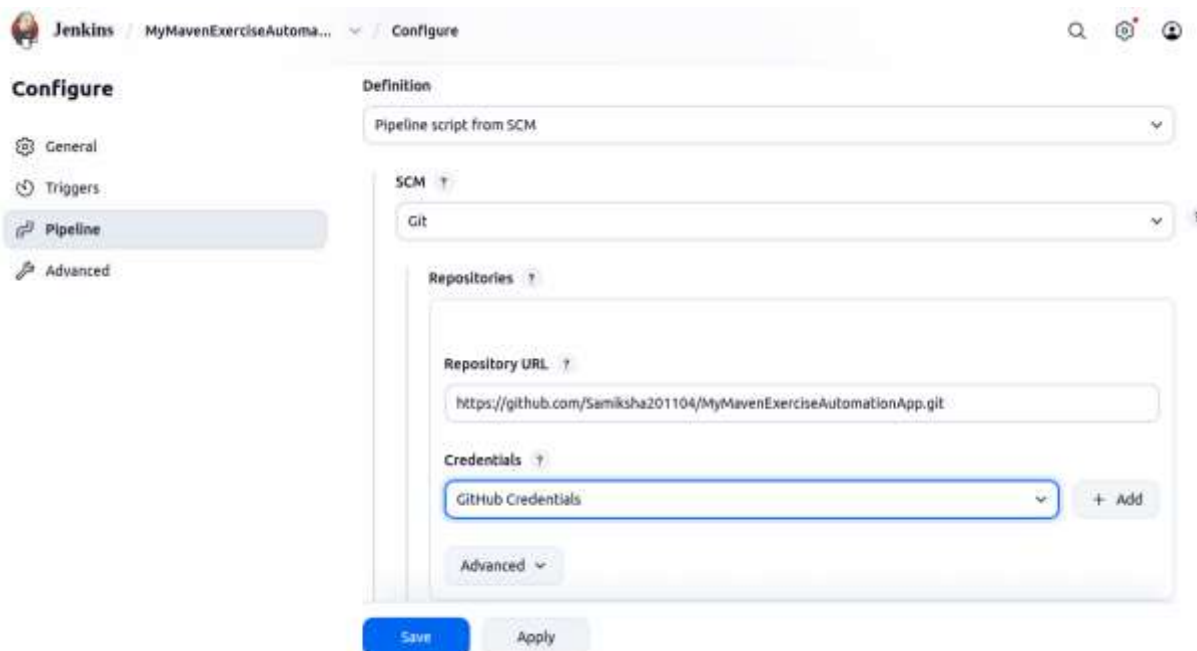
Then commit and push it to GitHub

Step 2:in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



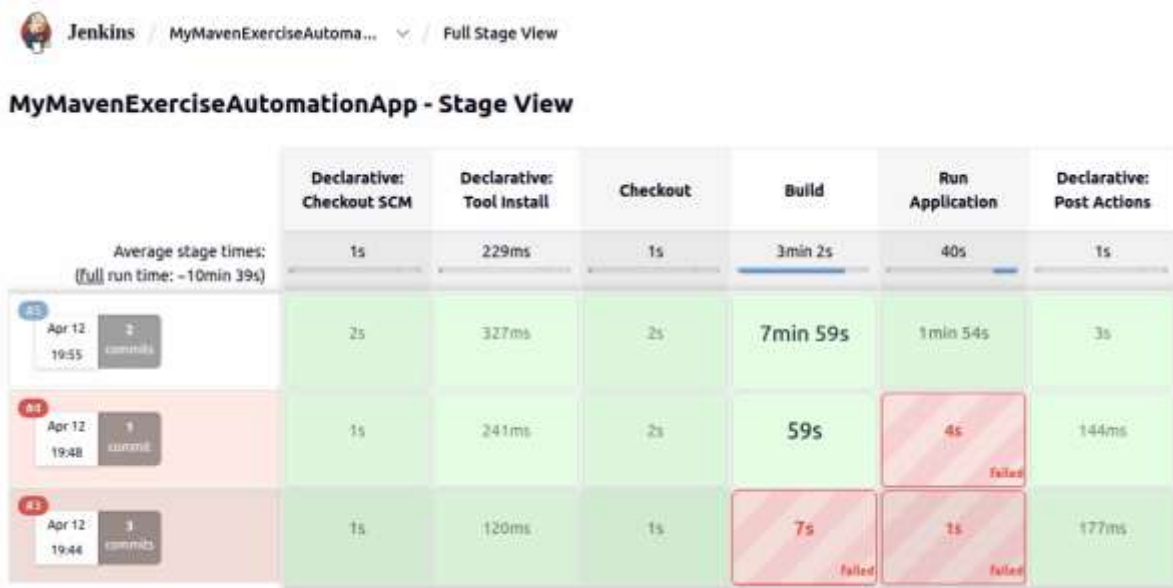
The image shows the Jenkins 'New Item' dialog. At the top, there's a breadcrumb 'Jenkins / All / New Item'. The title is 'New Item'. Below it, a text field 'Enter an Item name' contains 'MyMavenExerciseAutomationApp'. Under 'Select an item type', there are four options: 'Pipeline' (with a blue icon), 'Freestyle project' (with a cube icon), 'Maven project' (with a checkmark icon), and 'Multi-configuration project' (with a diamond icon). The 'Maven project' option is selected. At the bottom is an 'OK' button.

In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials



The image shows the Jenkins 'Configure' page for the item 'MyMavenExerciseAutomationApp'. The left sidebar has 'Configure' at the top, followed by 'General', 'Triggers', 'Pipeline' (which is selected and highlighted), and 'Advanced'. The main area is titled 'Definition' and shows 'Pipeline script from SCM' selected in a dropdown. Below this, 'SCM' is set to 'Git'. Under 'Repositories', there is a 'Repository URL' field containing 'https://github.com/Samiksha201104/MyMavenExerciseAutomationApp.git'. Below that, 'Credentials' is set to 'GitHub Credentials' with an '+ Add' button. At the bottom are 'Save' and 'Apply' buttons.

Step 3 : in the Dashboard click on the MyMavenExerciseAutomationApp item and select Build Now and then Full Stage view



MyMavenGuavaApp

Step 1: now open the terminal and type the command **cd MyMavenGuavaApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile pipeline {

```
agent any // Use any available agent
```

```
tools {
```

```
    maven 'Maven' // Ensure this matches the name configured in Jenkins
```

```
}
```

```
stages {
```

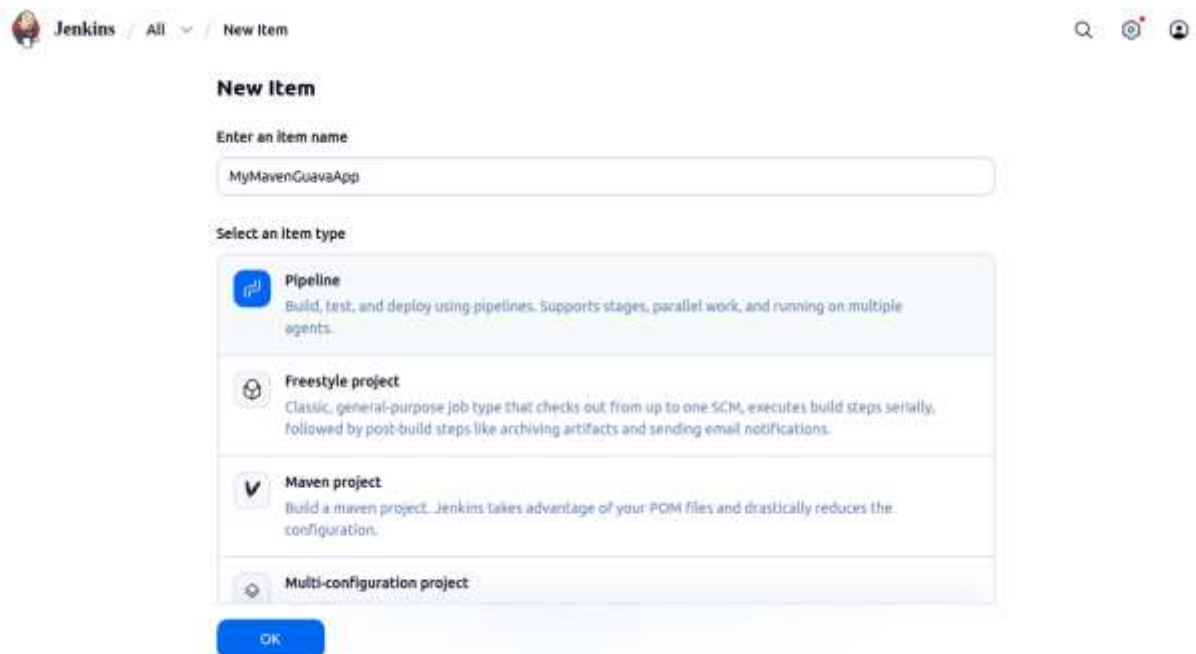
```
    stage('Checkout') {
```

```
        steps {
```

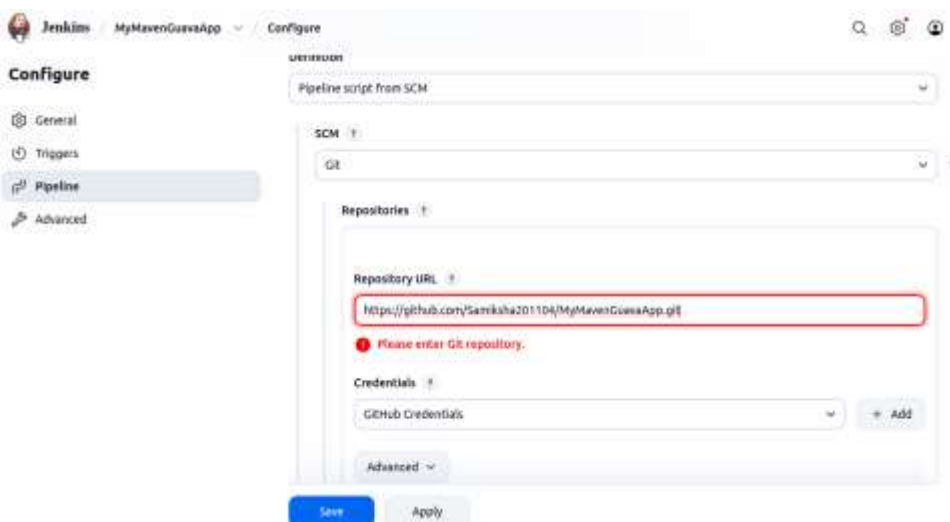
```
git          branch:          'master',          url:
'https://github.com/Samiksha201104/MyMavenGuavaApp.git'
    }
  }
  stage('Build') {
    steps {
      sh 'mvn clean package' // Run Maven build
    }
  }
  stage('Test') {
    steps {
      sh 'mvn test' // Run unit tests
    }
  }
  stage('Run Application') {
    steps {
      // Start the JAR application
      sh 'java -jar target/MyMavenGuavaApp-1.0-SNAPSHOT.jar'
    }
  }
}
post {
  success {
    echo 'Build and deployment successful!'
  }
  failure {
    echo 'Build failed!'
  }
}}
```

Then commit and push it to GitHub

Step 2: in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials



Step 3 : in the Dashboard click on the MyMavenGuavaApp item and select Build Now and then Full Stage view

Correct the errors and build again on jenkins till all the stages are successful



MyMavenToGradle

Step 1: now open the terminal and type the command `cd MyMavenToGradleApp` to enter the folder and create a **Jenkinsfile**. Paste the code in the Jenkinsfile

pipeline {

agent any // Use any available agent

tools {

maven 'Maven' // Ensure this matches the name configured in Jenkins

}

stages {

stage('Checkout') {

steps {

git branch: 'master', url:

'https://github.com/Samiksha201104/MyMavenGradleApp.git'

}

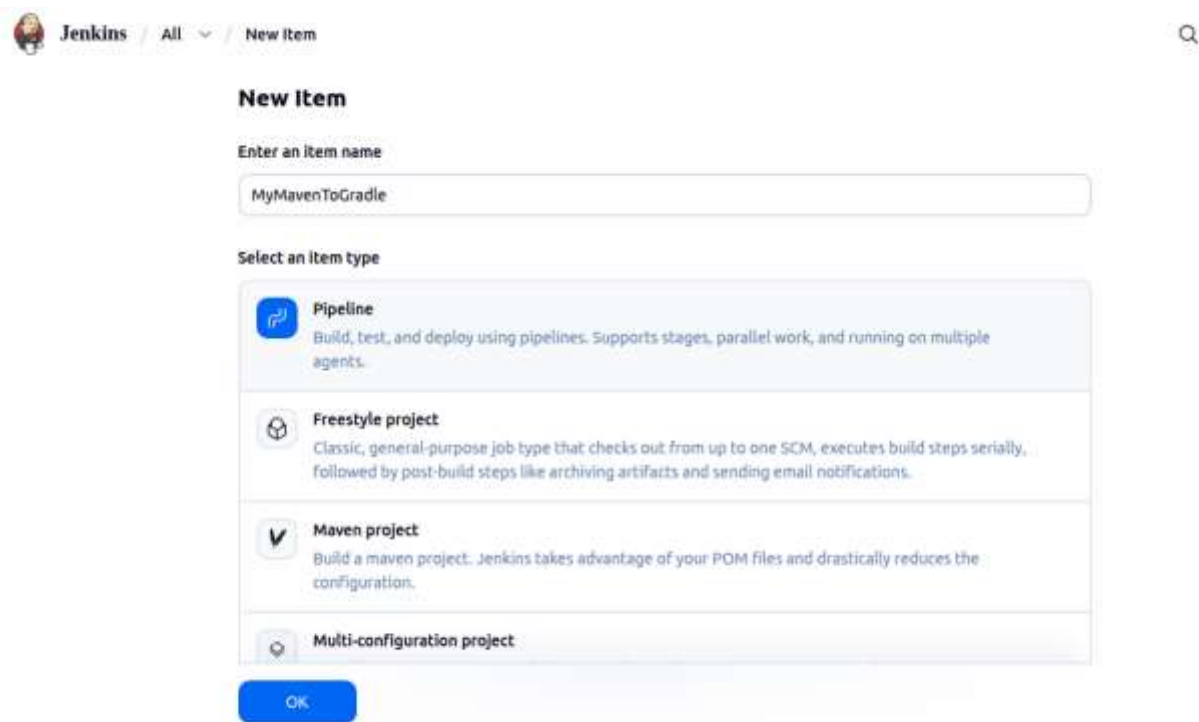
}

stage('Build') {

```
    steps {
        sh 'mvn clean package' // Run Maven build
    }
}
stage('Test') {
    steps {
        sh 'mvn test' // Run unit tests
    }
}
stage('Run Application') {
    steps {
        // Start the JAR application
        sh 'java -jar target/MyMavenGradleApp-1.0-SNAPSHOT.jar'
    }
}
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}}
```

Then commit and push it to GitHub

Step 2:in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



Jenkins / All / New Item

New Item

Enter an item name

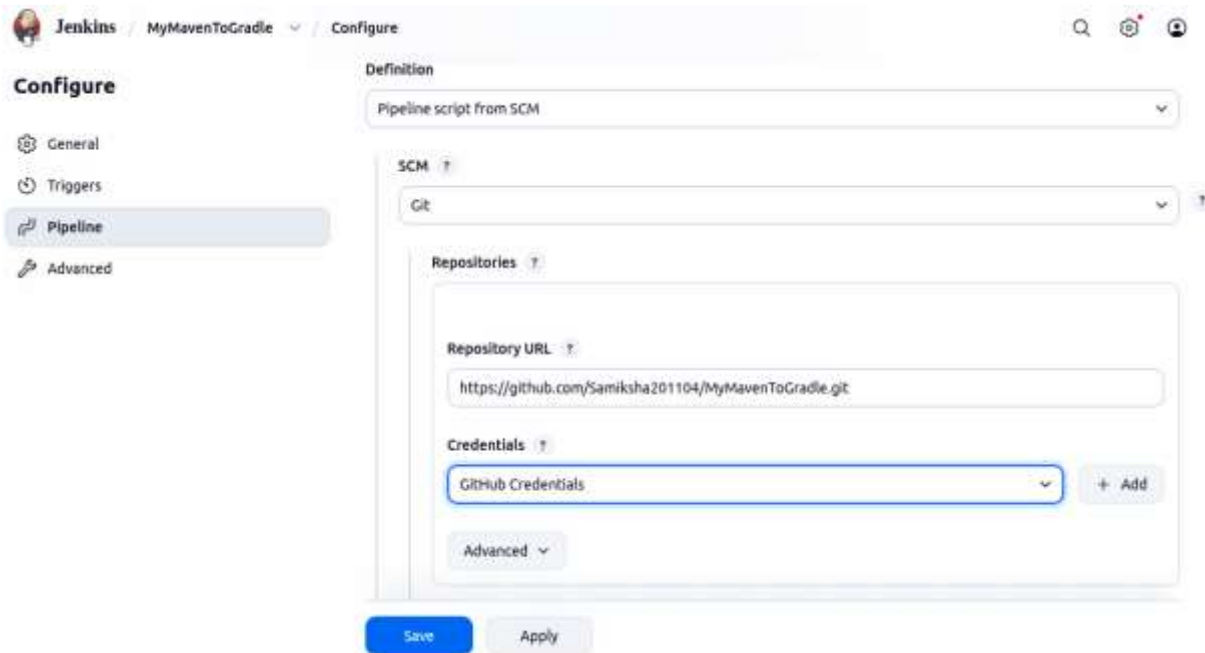
MyMavenToGradle

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project** (Selected)
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**

OK

In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials



Jenkins / MyMavenToGradle / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/Samiksha201104/MyMavenToGradle.git

Credentials

GitHub Credentials

+ Add

Advanced

Save Apply

Step 3 : in the Dashboard click on the MyMavenToGradleApp item and select Build Now and then Full Stage view

Correct the errors and build again on Jenkins till all the stages are successful



MyMavenSeleniumApp

Step 1: now open the terminal and type the command **cd MyMavenSeleniumApp** to enter the folder and create a **Jenkinsfile**. Paste the code in the Jenkinsfile

pipeline {

agent any // Use any available agent

tools {

maven 'Maven' // Ensure this matches the name configured in Jenkins

}

stages {

stage('Checkout') {

steps {

git branch: 'master', url:

'https://github.com/Samiksha201104/MyMavenSeleniumApp.git'

}

}

stage('Build') {

steps {

sh 'mvn clean package' // Run Maven build

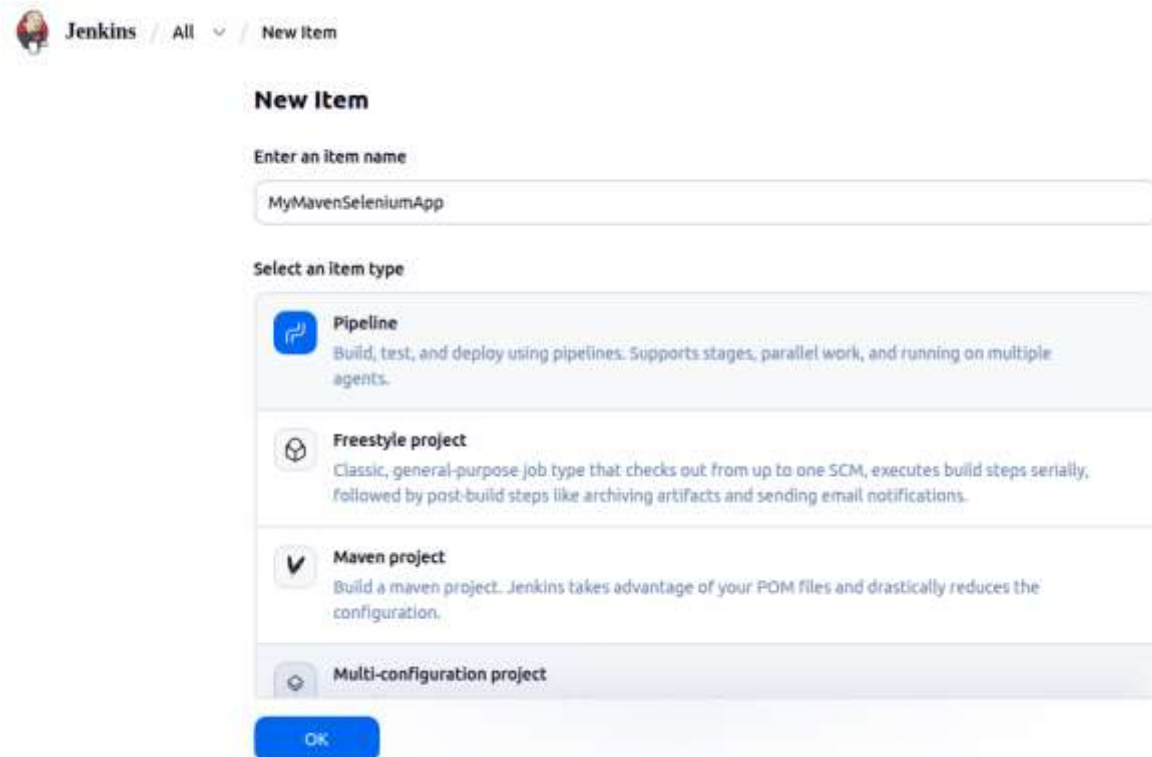
}

}

```
stage('Test') {  
    steps {  
        sh 'mvn test' // Run unit tests  
    }  
}  
  
stage('Run Application') {  
    steps {  
        // Start the JAR application  
        sh 'java -jar target/MyMavenSeleniumApp-1.0-SNAPSHOT.jar'  
    }  
}  
}  
  
post {  
    success {  
        echo 'Build and deployment successful!'  
    }  
    failure {  
        echo 'Build failed!'  
    }  
}}
```

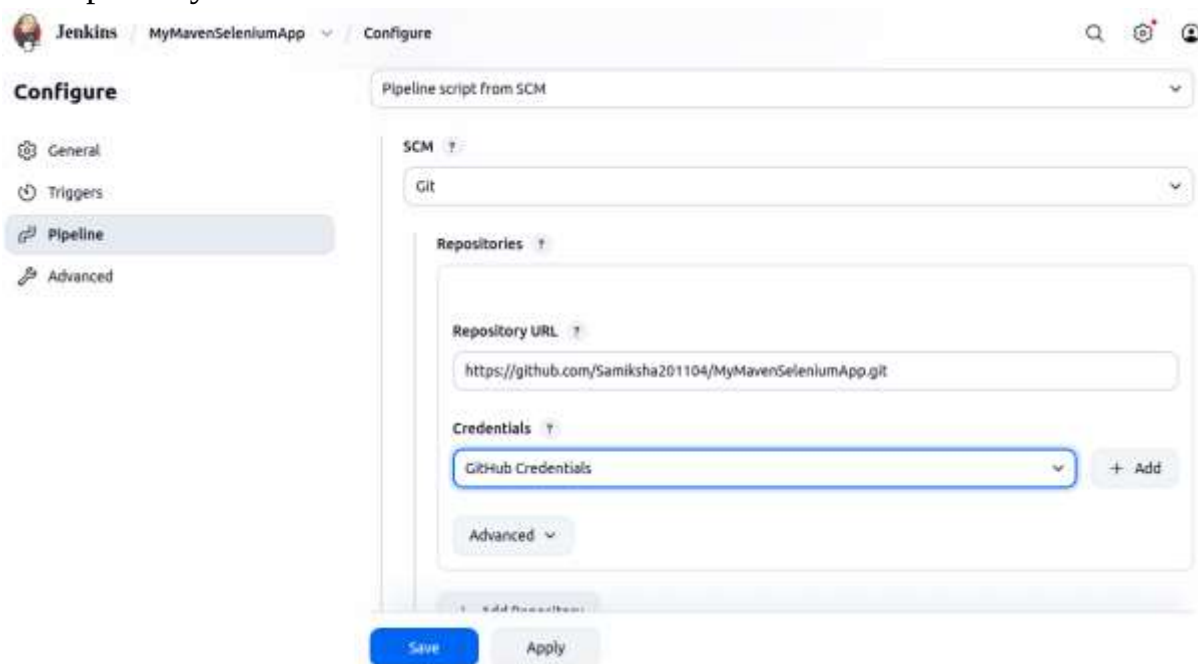
Then commit and push it to GitHub

Step 2:in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



The image shows the 'New Item' dialog in Jenkins. At the top, there's a breadcrumb 'Jenkins / All / New Item'. The title is 'New Item'. Below it, a text field 'Enter an item name' contains 'MyMavenSeleniumApp'. Under 'Select an item type', there are four options: 'Pipeline' (selected with a blue icon), 'Freestyle project', 'Maven project' (marked with a checkmark), and 'Multi-configuration project'. Each option has a brief description. At the bottom is a blue 'OK' button.

In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credentials



The image shows the 'Configure' page for the 'MyMavenSeleniumApp' item. The breadcrumb is 'Jenkins / MyMavenSeleniumApp / Configure'. On the left, there's a sidebar with 'General', 'Triggers', 'Pipeline' (selected), and 'Advanced'. The main area has a dropdown 'Pipeline script from SCM'. Below it, 'SCM' is set to 'Git'. Under 'Repositories', there's a 'Repository URL' field with 'https://github.com/Samiksha201104/MyMavenSeleniumApp.git'. Below that, 'Credentials' is set to 'GitHub Credentials' with a '+ Add' button. There's an 'Advanced' dropdown and a 'Add new repository' link. At the bottom are 'Save' and 'Apply' buttons.

Step 3 : in the Dashboard click on the MyMavenToGradleApp item and select Build Now and then Full Stage view

Correct the errors and build again on jenkins till all the stages are successful



Working With Selenium Using firefox

All 3 exercises executed using firefox

Step 1: create a maven project on terminal by typing the command `mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenFireFox -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false`

```
sanikshageantksha: /DevOps02$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenFireFox -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.456 s
[INFO] Finished at: 2026-03-17T20:40:07+05:30
[INFO] -----
```

Step 2 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>MyMavenExerciseAutomationApp</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>MyMavenExerciseAutomationApp</name>

  <!-- Java Version -->
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>

    <!-- Selenium -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.29.0</version>
```

```
</dependency>
```

```
<!-- WebDriverManager (handles GeckoDriver automatically) -->
```

```
<dependency>
```

```
  <groupId>io.github.bonigarcia</groupId>
```

```
  <artifactId>webdrivermanager</artifactId>
```

```
  <version>5.8.0</version>
```

```
</dependency>
```

```
<!-- JUnit (optional, for testing) -->
```

```
<dependency>
```

```
  <groupId>junit</groupId>
```

```
  <artifactId>junit</artifactId>
```

```
  <version>4.13.2</version>
```

```
  <scope>test</scope>
```

```
</dependency>
```

```
</dependencies>
```

```
<build>
```

```
  <plugins>
```

```
    <!-- Compiler Plugin -->
```

```
    <plugin>
```

```
      <artifactId>maven-compiler-plugin</artifactId>
```

```
      <version>3.11.0</version>
```

```
    </plugin>
```

```
<!-- Exec Plugin (to run your main class) -->
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>exec-maven-plugin</artifactId>
  <version>3.1.0</version>
  <configuration>
    <mainClass>com.example.App</mainClass>
  </configuration>
</plugin>

</plugins>
</build>
```

```
</project>
```

Step 3: Edit the App.java

```
package com.example;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.firefox.FirefoxDriver;
```

```
public class App {
```

```
    public static void main(String[] args) {
```

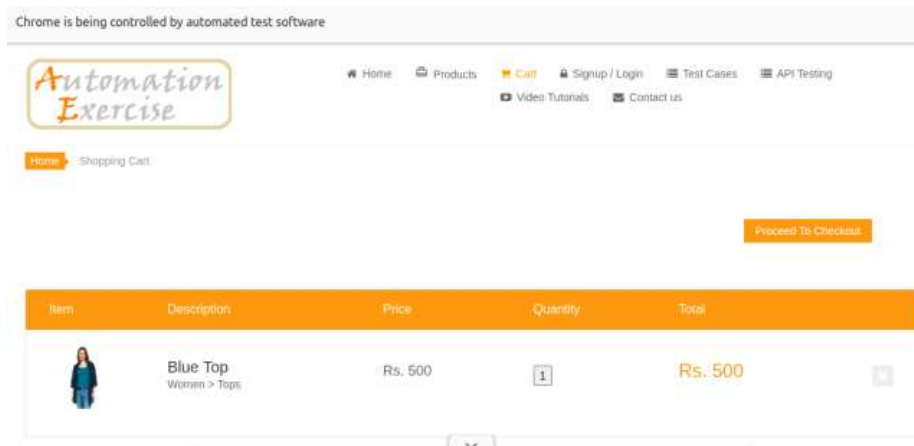
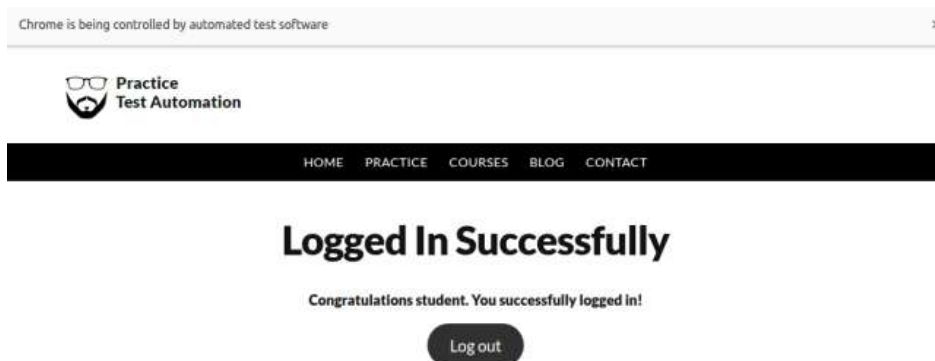
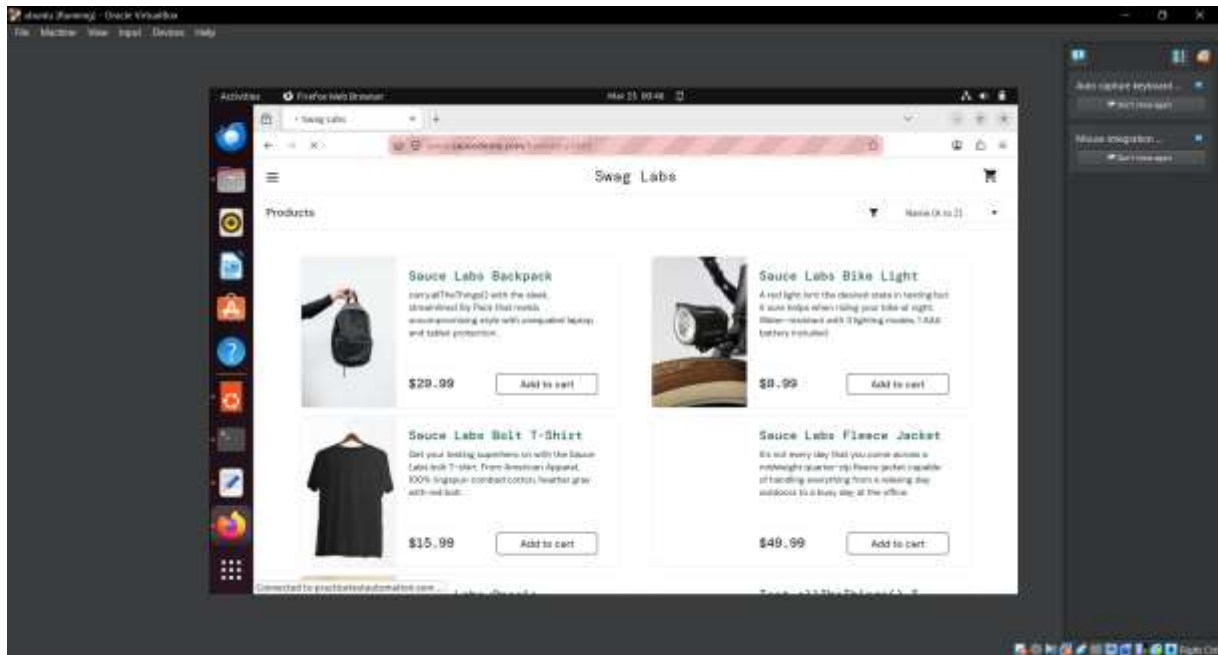
```
        WebDriver driver = new FirefoxDriver();
```

```
// ----- 1. SauceDemo Login -----  
driver.get("https://www.saucedemo.com/");  
driver.manage().window().maximize();  
  
driver.findElement(By.id("user-name")).sendKeys("standard_user");  
driver.findElement(By.id("password")).sendKeys("secret_sauce");  
driver.findElement(By.id("login-button")).click();  
  
// ----- 2. Practice Test Login -----  
driver.get("https://practicetestautomation.com/practice-test-login/");  
driver.manage().window().maximize();  
  
driver.findElement(By.id("username")).sendKeys("student");  
driver.findElement(By.id("password")).sendKeys("Password123");  
driver.findElement(By.id("submit")).click();  
  
// ----- 3. Automation Exercise Search -----  
driver.get("https://automationexercise.com/products");  
driver.manage().window().maximize();  
  
driver.findElement(By.id("search_product")).sendKeys("Blue Top");  
driver.findElement(By.id("submit_search")).click();  
  
// driver.quit(); // optional  
}
```

Step 4: build and run the maven application

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**

This executes all the three exercises together.



Installation of Apache Tomcat and Jenkins Pipeline for MyMavenWebApp01

Installation of Apache Tomcat:

Step 1: To install Apache tomcat in root folder, type the command **sudo -i** and then **sudo apt update && sudo upgrade -y**

```
samiksha@samiksha:~$ sudo -i
[sudo] password for samiksha:
root@samiksha:~# sudo apt update && sudo upgrade -y
Hit:1 https://dl.google.com/linux/chrome/deb stable InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://ln.archive.ubuntu.com/ubuntu jammy InRelease
Hit:4 http://ln.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 http://ln.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
78 packages can be upgraded. Run 'apt list --upgradable' to see them.
sudo: upgrade: command not found
```

Enter tmp by typing the command **cd /tmp** and download Apache Tomcat software by typing the command **wget**

<https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz>

```
root@samiksha:~# cd /tmp
root@samiksha:/tmp# wget https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
--2026-04-14 15:46:37-- https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f8:1a:a004::2
Connecting to archive.apache.org (archive.apache.org)[65.108.204.189]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12715996 (12M) [application/x-gzip]
Saving to: 'apache-tomcat-9.0.95.tar.gz'

apache-tomcat-9.0.95.tar.gz 100%[=====] 12.13M 83.2KB/s in 1m 41s
2026-04-14 15:48:19 (123 KB/s) - 'apache-tomcat-9.0.95.tar.gz' saved [12715996/12715996]
```

Then enter type **cd ..** and extract the downloaded to opt in root by typing the commands

cd opt

mkdir tomcat

cd ..

cd tmp

to extract the tar file into tomcat folder run the command **sudo tar -xvzf apache-tomcat-9.0.95.tar.gz -C /opt/tomcat --strip-components=1**


```

root@samiksha:/tmp# cd ..
root@samiksha:/# cd opt
root@samiksha:/opt# cd tmp
-bash: cd: tmp: No such file or directory
root@samiksha:/opt# ls
google  gradle
root@samiksha:/opt# mkdir tomcat
root@samiksha:/opt# cd ..
root@samiksha:/# cd tmp
root@samiksha:/tmp# sudo tar -xvzf apache-tomcat-9.0.95.tar.gz -C /opt/tomcat --strip-components=1
apache-tomcat-9.0.95/conf/
apache-tomcat-9.0.95/conf/catalina.policy
apache-tomcat-9.0.95/conf/catalina.properties
apache-tomcat-9.0.95/conf/context.xml
apache-tomcat-9.0.95/conf/jaspic-providers.xml
apache-tomcat-9.0.95/conf/jaspic-providers.xsd
apache-tomcat-9.0.95/conf/logging.properties
apache-tomcat-9.0.95/conf/server.xml
apache-tomcat-9.0.95/conf/tomcat-users.xml

```

Enter tomcat and type **ls** to see all files are extracted

Enter conf **cd conf** and type ls to see server.

```

root@samiksha:/tmp# cd ..
root@samiksha:/# cd opt
root@samiksha:/opt# cd tomcat/
root@samiksha:/opt/tomcat# ls
bin  BUILDING.txt  conf  CONTRIBUTING.md  lib  LICENSE  logs  NOTICE  README.md  RELEASE-NOTES  RUNNING.txt  temp  webapps  work
root@samiksha:/opt/tomcat#
root@samiksha:/opt/tomcat# cd conf
root@samiksha:/opt/tomcat/conf# ls
catalina.policy      context.xml      jaspic-providers.xsd  server.xml      tomcat-users.xsd
catalina.properties  jaspic-providers.xml  logging.properties  tomcat-users.xml  web.xml
root@samiksha:/opt/tomcat/conf#

```

xml use this file to change the port number from default 8080 to any number say 9090 because Jenkins is running on 8080

```
root@samiksha:/opt/tomcat/conf# gedit server.xml
```

Type **gedit server.xml** and go to line number 70 and change the portnumber to and save

```

-->
<Connector port="9090" protocol="HTTP/1.1"
    connectionTimeout="20000"
    redirectPort="8443"
    maxParameterCount="1000"
/>
<!-- A "Connector" using the shared thread pool-->
<!--
<Connector executor="tomcatThreadPool"
    port="8080" protocol="HTTP/1.1"
    connectionTimeout="20000"
    redirectPort="8443"
    maxParameterCount="1000"
/>

```

we need to create tomcat users as well so add the users in tomcat-users.xml file by typing the command **gedit tomcat-users.xml**

```
root@samiksha:/opt/tomcat/conf# gedit tomcat-users.xml
```

go to line 22 add the line **<user username="admin" password="Admin" roles="manager-gui"/>**

```
<tomcat-users xmlns="http://tomcat.apache.org/xml"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://tomcat.apache.org/xml tomcat-users.xsd"
  version="1.0">
  <!--
    <user username="admin" password="Admin" roles="manager-gui"/>
  -->
</tomcat-users>
```

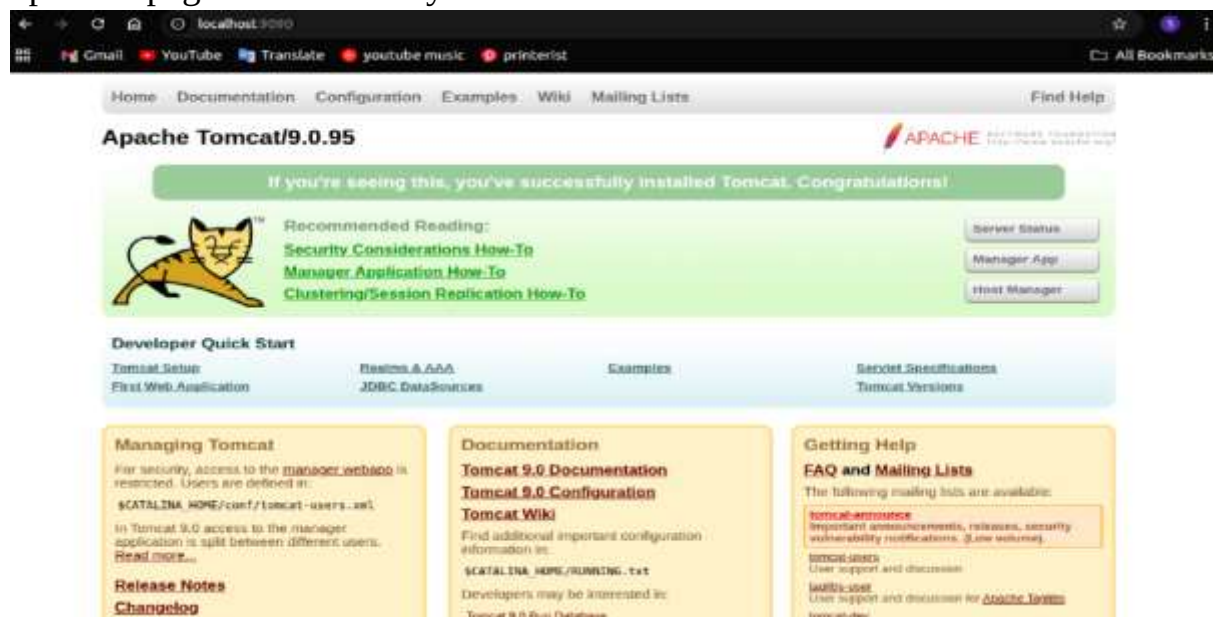
Then enter bin and type ls

```
root@samiksha:/opt/tomcat/conf# cd ..
root@samiksha:/opt/tomcat# cd bin
root@samiksha:/opt/tomcat/bin# ls
bootstrap.jar      ciphers.sh          daemon.sh            setclasspath.bat    startup.sh           version.bat
catalina.bat       commons-daemon.jar  digest.bat           setclasspath.sh     tomcat-juli.jar     version.sh
catalina.sh        commons-daemon-native.tar.gz  digest.sh           shutdown.bat        tomcat-native.tar.gz
catalina-tasks.xml configtest.bat       makebase.bat         shutdown.sh         tool-wrapper.bat
ciphers.bat        configtest.sh       makebase.sh          startup.bat         tool-wrapper.sh
```

we can type the commands **startup.sh** and **shutdown.sh** to start the tomcat and stop the tomcat

```
root@samiksha:/opt/tomcat/bin# sh startup.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
root@samiksha:/opt/tomcat/bin#
```

open the page in browser say localhost:9090



Click on Manager App enter the credentials previously provided



Tomcat Web Application Manager

Message:

OK

Manager

List Applications

HTML Manager Help

Manager Help

Server Status

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expires sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expires sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expires sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expires sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	3	Start Stop Reload Undeploy Expires sessions with idle ≥ 30 minutes

Jenkins Pipeline for MyMavenWebApp01

Step 1: open the terminal and type the command **cd MyMavenWebApp01** and create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

pipeline {

agent any // Use any available agent

tools {

maven 'Maven' // Ensure this matches the name configured in Jenkins

}

stages {

stage('Checkout') {

steps {

git branch: 'master', url: 'https://github.com/Samiksha201104/MyMavenWebApp01.git'

}}

stage('Build') {

```
    steps {
        sh 'mvn clean package' // Run Maven build
    }
stage('Test') {
    steps {
        sh 'mvn test' // Run unit tests
    }
stage('Deploy WAR') {
    steps {
        sh 'cp target/MymavenWebApp01.war /opt/tomcat/webapps/'
    }
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
```

Go to index.jsp file and edit the hello world code and paste the following code

```
1  <html>
2  <body>
3  <h2>Samiksha P 1BI23CS189</h2>
4  </body>
5  </html>
```

Step 2: Go to Jenkins Dashboard click on new item enter the item name as MyMavenWebApp and select pipeline. In the configure select pipeline scm paste the repository url select github credentials apply and save

The top screenshot shows the Jenkins 'New Item' page. The 'Enter an item name' field contains 'MyMavenWebApp'. Under 'Select an item type', the 'Maven project' option is selected with a checkmark. The description for 'Maven project' is 'Build a maven project; Jenkins takes advantage of your POM files and drastically reduces the configuration.'

The bottom screenshot shows the 'Configure' page for 'MyMavenWebApp'. The 'Pipeline' tab is selected in the left sidebar. The 'Pipeline script from SCM' dropdown is set to 'Pipeline script from SCM'. Under 'SCM', 'Git' is selected. In the 'Repositories' section, the 'Repository URL' is 'https://github.com/Samisha201104/MyMavenWebApp01.git' and the 'Credentials' dropdown is set to 'Github Credentials'. There is an '+ Add' button next to the credentials dropdown.

Step 3: go **MyMavenWebApp** in Jenkins and click **Build Now**; the build completes successfully.



Step 4: go to Tomcat Web Application Manager and select the path /MyMavenWebApp01




Tomcat Web Application Manager

Message: OK

Manager

List Applications HTML Manager Help Manager Help Server Status

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/MyMavenWebApp01	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

localhost:9090/MyMavenWebApp01/

Gmail YouTube Apache Tomca... Dashboard - Je...

Samiksha P 1BI23CS189

Apache Tomcat and Jenkins Pipeline for MavenAnsibleWebApp1 along with Ansible

Step 1: create a Web app by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MavenAnsibleWebApp1 -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false**

```
saniksha@saniksha:~$ cd Devops60
saniksha@saniksha:~/Devops60$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenAnsibleWebApp01 -DarchetypeArtifactId
maven-archetype-webapp -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO]
[INFO] -----
[INFO] Parameter: basedir, Value: /home/saniksha/Devops60
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenAnsibleWebApp01
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/saniksha/Devops60/MyMavenAnsibleWebApp01
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 4.383 s
[INFO] Finished at: 2026-04-15T16:30:01+05:30
[INFO] -----
```

Step 2 : enter the MavenAnsibleWebApp1 , open the pom.xml file and paste the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId>
```

```
<artifactId>MavenAnsibleWebApp1</artifactId>
```

```
<packaging>war</packaging>
```

```
<version>1.0-SNAPSHOT</version>
```

```
<name>MavenAnsibleWebApp1</name>
```

```
<url>http://maven.apache.org</url>
```

```
<dependencies>
```

```
<dependency>
```

```
<groupId>junit</groupId>
```

```
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
<!-- Servlet API for Java EE -->
<dependency>
<groupId>javax.servlet</groupId>
<artifactId>javax.servlet-api</artifactId>
<version>4.0.1</version>
<scope>provided</scope>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-war-plugin</artifactId>
<version>3.4.0</version>
<!-- Update to latest stable version -->
</plugin>
</plugins>
<finalName>MavenAnsibleWebApp1</finalName>
<!-- create a web page with the name MavenAnsibleWebApp, if we want some
other name we can change it -->
</build>
</project>
```

Step 3: create Jenkinsfile by typing the command **gedit Jenkinsfile** and paste the code


```
pipeline {
  agent any // Use any available agent
  environment {
    LANG = 'en_US.UTF-8'
    LC_ALL = 'en_US.UTF-8'
  } // this has to be added only if you get an error saying UTF required is 8 but
  showing in ISO00009  tools {
    maven 'Maven' // Ensure this matches the name configured in Jenkins
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'master', url:
'https://github.com/ShruthiBGowda/MavenAnsibleWebApp1-CICD.git'
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean package' // Run Maven build
      }
    }
    stage('Archive') {
      steps {
        archiveArtifacts artifacts: 'target/*.war', fingerprint:true
      }
    }
    stage('Deploy') {
      steps {
        sh 'mvn clean package'
        sh 'ansible-playbook ansible/playbook.yml -i ansible/hosts.ini'
      }
    }
  }
}
```

Step 4: create a new directory by typing the command **mkdir ansible** . Inside ansible create the hosts.ini file and playbook.yml file.

Paste the following code in **hosts.ini**

[web]

localhost ansible_connection=local
ansible_python_interpreter=/usr/bin/python3

and paste this code in **playbook.yml**

- name: Deploy to Tomcat

hosts: web

become: yes

tasks:

- name: Copy WAR file to Tomcat

copy:

src: "/var/snap/jenkins/5013/workspace/MavenAnsibleWebApp1-CICD/target/MavenAnsibleWebApp1.war"

dest: "/opt/tomcat/webapps/MavenAnsibleWebApp1.war"

remote_src: no

- name: Stop Tomcat

shell: /opt/tomcat/bin/shutdown.sh

ignore_errors: yes

- name: Start Tomcat

shell: /opt/tomcat/bin/startup.sh

```
samiksha@samiksha:~/Devops6D/MavenAnsibleWebApp1$ tree
```

```

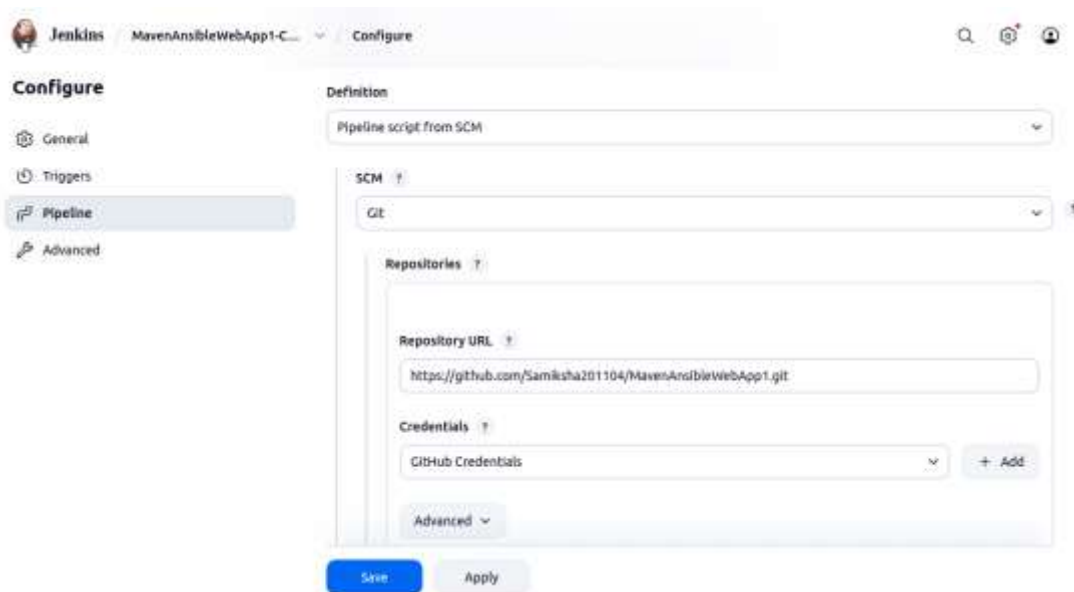
.
├── Jenkinsfile
├── ansible
│   ├── hosts.ini
│   └── playbook.yml
├── pom.xml
├── src
│   └── main
│       ├── resources
│       └── webapp
│           ├── WEB-INF
│           │   └── web.xml
│           └── index.jsp

```

7 directories, 6 files

Commit everything and push it to GitHub

Step 5: Go to Jenkins Dashboard click on new item and give the name MavenAnsibleWebApp1-CICD and select pipeline. In the configure select pipeline with scm paste the GitHub url and select git credentials apply and save .



Step 6: Go back to dashboard and build MavenAnsibleWebApp1-CICD



Step 7: Start Tomcat by typing the command **sh startup.sh**

```
samiksha@samiksha:~$ sudo -i
[sudo] password for samiksha:
root@samiksha:~# cd /
root@samiksha:~# cd /opt
root@samiksha:/opt# cd tomcat/
root@samiksha:/opt/tomcat# cd bin
root@samiksha:/opt/tomcat/bin# sh startup.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
root@samiksha:/opt/tomcat/bin#
```

Step 8: Login to Manager app using the credentials given and select the path /MavenAnsibleWebApp1



Tomcat Web Application Manager

Message:

OK

Manager

List Applications

HTML Manager Help

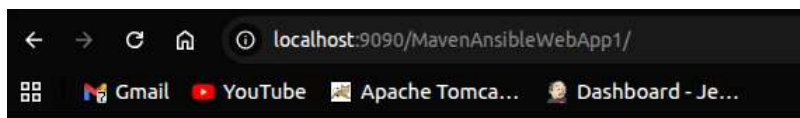
Manager Help

Server Status

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/MavenAnsibleWebApp1	None specified	Archetype Created Web Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/MymavenWebApp01	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

We will get hello world in the browser



Hello World!

Program 7:

Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle

Step 1: To build a basic Java Application with Maven Project, run following command:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMaventoGradle -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
saniksha@saniksha:~/Devops60$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMaventoGradle -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> mvn archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< mvn archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- mvn archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/saniksha/Devops60
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMaventoGradle
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/saniksha/Devops60/MyMaventoGradle
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 6.666 s
[INFO] Finished at: 2026-03-25T11:57:45+05:30
[INFO]
[INFO] -----
```

Step 2: enter the MyMaventoGradle directory and open the pom.xml file.

```
saniksha@saniksha:~/Devops60$ cd MyMaventoGradle
saniksha@saniksha:~/Devops60/MyMaventoGradle$ ls
pom.xml  src
saniksha@saniksha:~/Devops60/MyMaventoGradle$ gedit pom.xml
```

Add the code in the pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>MyMavenApp</artifactId>
<packaging>jar</packaging>
<version>1.0-SNAPSHOT</version>
<name>MyMavenApp</name>
<url>http://maven.apache.org</url>

<!-- Properties: Customize Java version or plugin versions -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
```

```
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-jar-plugin</artifactId>
    <version>3.1.0</version>
    <configuration>
        <archive>
            <manifestEntries>
                <Main-Class>com.example.MainClass</Main-Class>
            </manifestEntries>
        </archive>
    </configuration>
</plugin>
</plugins>
</build>
```

</project>

Step 3 : Run **mvn clean install** to compile your project

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.051 s - in com.example.AppTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenApp ---
[INFO] Building jar: /home/samiksha/Devops6D/MyMavenToGradle/target/MyMavenApp-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenApp ---
[INFO] Installing /home/samiksha/Devops6D/MyMavenToGradle/target/MyMavenApp-1.0-SNAPSHOT.jar to /home/samiksha/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/samiksha/Devops6D/MyMavenToGradle/pom.xml to /home/samiksha/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.933 s
[INFO] Finished at: 2026-03-25T12:07:52+05:30
[INFO] -----
```

Step 4: Type the command **mvn exec:java -Dexec.mainClass="com.example.App"** to execute

```
samiksha@samiksha:~/Devops6D/MyMavenToGradle$ mvn exec:java -Dexec.mainClass="com.example.App"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenApp >-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ MyMavenApp ---
Hello World!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.662 s
[INFO] Finished at: 2026-03-25T12:10:12+05:30
[INFO] -----
samiksha@samiksha:~/Devops6D/MyMavenToGradle$
```

Step 5: Type the command **gradle init --type pom** to initialize a Gradle Project using gradle init command with type as pom, command converts pom.xml structure to Gradle's build.gradle.

```
samiksha@samiksha:~/Devops6D/MyMavenToGradle$ gradle init --type pom
Select build script DSL:
  1: Kotlin
  2: Groovy
Enter selection (default: Kotlin) [1..2] 2

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] no

> Task :init
Maven to Gradle conversion is an incubating feature.
For more information, please refer to https://docs.gradle.org/8.7/userguide/migrating_from_maven.html in the Gradle documentation.

BUILD SUCCESSFUL in 1m 54s
1 actionable task: 1 executed
samiksha@samiksha:~/Devops6D/MyMavenToGradle$
```

Type the **ls** command to see all the files and directories.

```
samiksha@samiksha:~/Devops6D/MyMaventoGradle$ ls
build.gradle  gradle  gradlew  gradlew.bat  pom.xml  settings.gradle  src  target
samiksha@samiksha:~/Devops6D/MyMaventoGradle$ █
```

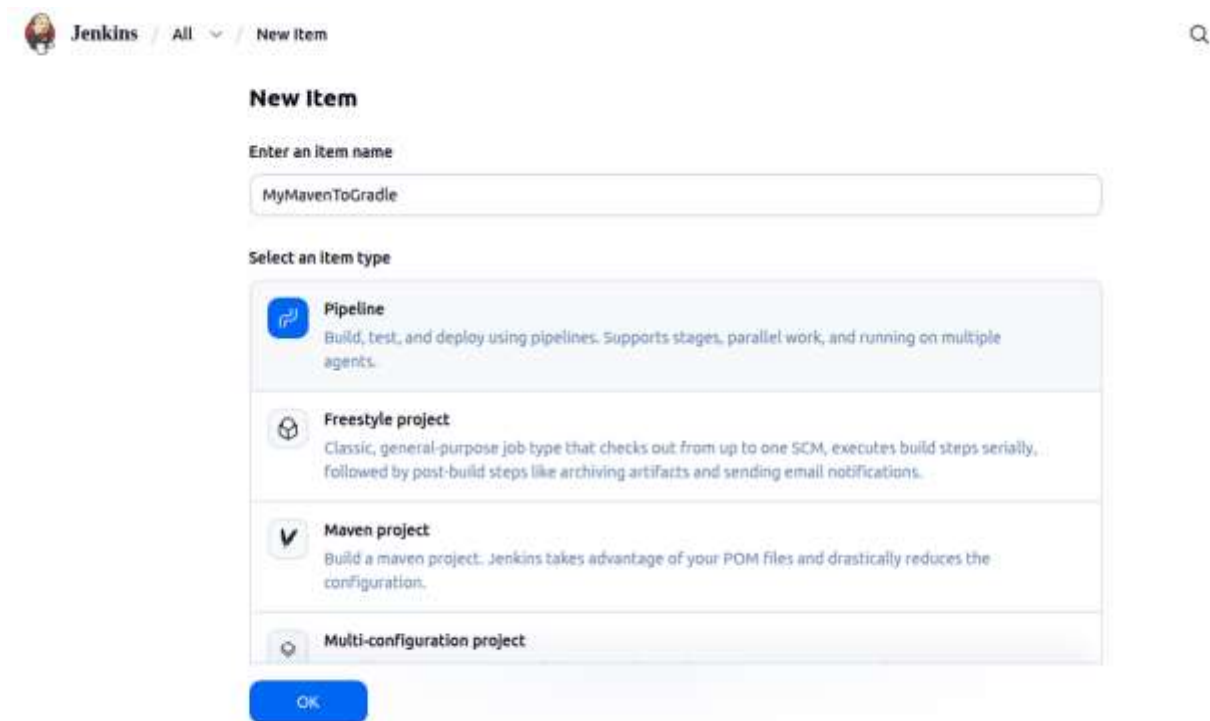
Step 6: now open the terminal and type the command **cd MyMavenToGradleApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

```
pipeline {
    agent any // Use any available agent
    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/Samiksha201104/MyMavenGradleApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package' // Run Maven build
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test' // Run unit tests
            }
        }
        stage('Run Application') {
            steps {
```

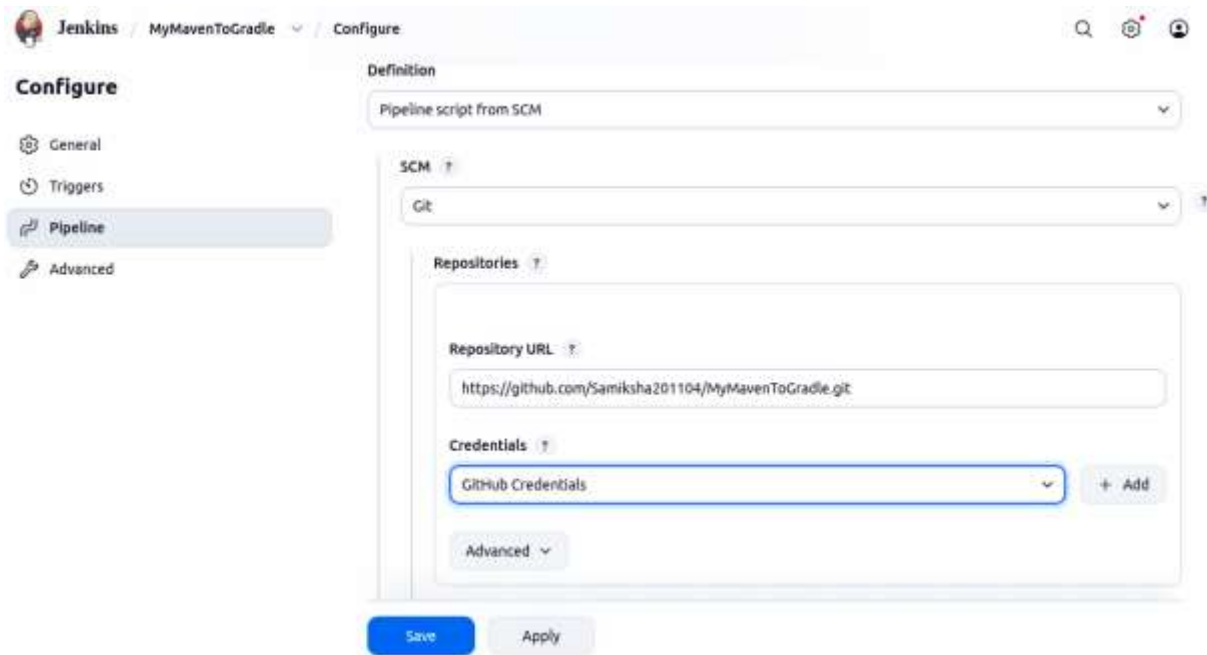
```
// Start the JAR application
sh 'java -jar target/MyMavenGradleApp-1.0-SNAPSHOT.jar'
}
}
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
```

Then commit and push it to GitHub

Step 7:in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credentials



Step 8 : in the Dashboard click on the MyMavenToGradleApp item and select Build Now and then Full Stage view

Correct the errors and build again on jenkins till all the stages are successful



Creating MavenAnsibleWebApp1

Step 1: creates a new ansible web application project named **MavenAnsibleWebApp1** using the **maven-archetype-webapp** code.

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MavenAnsibleWebApp1 -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
```

```
santiksha@santiksha:~$ cd Devops60
santiksha@santiksha:~/Devops60$ mvn archetype:generate -DgroupId=com.example -DartifactId=MavenAnsibleWebApp1 -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/santiksha/Devops60
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MavenAnsibleWebApp1
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/santiksha/Devops60/MavenAnsibleWebApp1
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.140 s
[INFO] Finished at: 2026-05-11T00:18:44+05:30
[INFO]
santiksha@santiksha:~/Devops60$
```

Step 2: Configure Maven Ansible Web App Project (pom.xml)

In this step, we move to the project directory using `cd MavenAnsibleWebApp` and open the Maven configuration file using `gedit pom.xml`. Then, we write the

required project details, dependencies, WAR packaging settings, and build configuration for creating the web application.

```

samiksha@samiksha:~/Devops6D$ cd MavenAnsibleWebApp1
samiksha@samiksha:~/Devops6D/MavenAnsibleWebApp1$ gedit pom.xml

```

pom.xml

```

<projectxmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/mavenv4_0_0.xsd">

<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>MavenAnsibleWebApp1</artifactId>
<packaging>war</packaging>
<version>1.0-SNAPSHOT</version>
<name>MavenAnsibleWebApp1</name>
<url>http://maven.apache.org</url>
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
<!-- Servlet API for Java EE -->
<dependency>
<groupId>javax.servlet</groupId>
<artifactId>javax.servlet-api</artifactId>

```

```
<version>4.0.1</version>
<scope>provided</scope>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-war-plugin</artifactId>
<version>3.4.0</version>
<!-- Update to latest stable version -->
</plugin>
</plugins>
<finalName>MavenAnsibleWebApp1</finalName>
<!-- create a web page with the name MavenAnsibleWebApp, if we want some
other name we can change it -->
</build>
</project>
```

Step 3: Create Ansible Configuration Files

In this step, we create a new folder named ansible inside the project directory using `mkdir ansible`. Then, we move into the folder using `cd ansible` and create the required Ansible files using `gedit hosts.ini` and `gedit playbook.yml`. These files are used to define target hosts and deployment tasks for automation.

```
samiksha@samiksha:~/Devops6D/MavenAnsibleWebApp1$ mkdir ansible
samiksha@samiksha:~/Devops6D/MavenAnsibleWebApp1$ cd ansible
samiksha@samiksha:~/Devops6D/MavenAnsibleWebApp1/ansible$ gedit hosts.ini
samiksha@samiksha:~/Devops6D/MavenAnsibleWebApp1/ansible$ gedit playbook.yml
samiksha@samiksha:~/Devops6D/MavenAnsibleWebApp1/ansible$ █
```

gedit hosts.ini

This command is written inside hosts.ini file to define the Ansible inventory. It creates a group named web and adds localhost as the target system, using local connection and Python 3 interpreter for executing Ansible tasks.

[web] localhost ansible_connection=local

ansible_python_interpreter=/usr/bin/python3

gedit playbook.yml

This code is written inside playbook.yml file to automate deployment using Ansible. It targets the web host group, copies the generated WAR file from the Jenkins workspace to the Tomcat webapps folder, and then restarts the Tomcat service so the updated application runs successfully.

- name: Deploy to Tomcat hosts: web

become: yes

tasks:

- name: Copy WAR file to Tomcat

copy:

src:

"/var/snap/jenkins/5013/workspace/MavenAnsibleWe

bApp1CICD/target/MavenAnsibleWebApp1.war"

dest:

"/opt/tomcat/webapps/MavenAnsibleWebApp1.war"

remote_src: no

- name: Stop Tomcat

shell: /opt/tomcat/bin/shutdown.sh

ignore_errors: yes

- name: Start Tomcat

shell: /opt/tomcat/bin/startup.shStep 4:

Build the Maven Ansible Web Application

In this step, we run the command `mvn clean install` to clean previous files and build the project.

```
samiksha@samiksha:~/Devops6D/MavenAnsibleWebApp1$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MavenAnsibleWebApp1 >-----
[INFO] Building MavenAnsibleWebApp1 1.0-SNAPSHOT
[INFO] -----[ war ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MavenAnsibleWebApp1 ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MavenAnsibleWebApp1 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ MavenAnsibleWebApp1 ---
[INFO] No sources to compile
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MavenAnsibleWebApp1 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/samiksha/Devops6D/MavenAnsibleWebApp1/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ MavenAnsibleWebApp1 ---
[INFO] No sources to compile
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ MavenAnsibleWebApp1 ---
[INFO] No tests to run.
[INFO]
[INFO] --- maven-war-plugin:3.4.0:war (default-war) @ MavenAnsibleWebApp1 ---
[INFO] Packaging webapp
[INFO] Assembling webapp [MavenAnsibleWebApp1] in [/home/samiksha/Devops6D/MavenAnsibleWebApp1/target/MavenAnsibleWebApp1]
[INFO] Processing war project
[INFO] Copying webapp resources [/home/samiksha/Devops6D/MavenAnsibleWebApp1/src/main/webapp]
[INFO] Building war: /home/samiksha/Devops6D/MavenAnsibleWebApp1/target/MavenAnsibleWebApp1.war
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MavenAnsibleWebApp1 ---
[INFO] Installing /home/samiksha/Devops6D/MavenAnsibleWebApp1/target/MavenAnsibleWebApp1.war to /home/samiksha/.m2/repository/com/example/MavenAnsibleWebApp1/1.0-SNAPSHOT/MavenAnsibleWebApp1-1.0-SNAPSHOT.war
[INFO] Installing /home/samiksha/Devops6D/MavenAnsibleWebApp1/pom.xml to /home/samiksha/.m2/repository/com/example/MavenAnsibleWebApp1/1.0-SNAPSHOT/MavenAnsibleWebApp1-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
```


Step 5: Verify Project Structure

In this step, we run the tree command to display the complete project structure.

```

.
├── ansible
│   ├── hosts.ini
│   └── playbook.yml
├── Jenkinsfile
├── pom.xml
├── src
│   └── main
│       ├── resources
│       └── webapp
│           ├── index.jsp
│           └── WEB-INF
│               └── web.xml
├── target
│   ├── classes
│   ├── MavenAnsibleWebApp1
│   │   ├── index.jsp
│   │   ├── META-INF
│   │   └── WEB-INF
│   │       └── classes
│   │           ├── web.xml
│   │           └── MavenAnsibleWebApp1.war
│   ├── maven-archiver
│   └── pom.properties
14 directories, 10 files
  
```

Setting Up Jenkins Pipeline for MavenAnsibleWebApp

Step 1: To deploy **MavenAnsibleWebApp1**, open the terminal, cd into the

MavenAnsibleWebApp1 folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

Edit Jenkinsfile:

```

pipeline {
    agent any

    // Use any available

    agent environment

    {

        LANG = 'en_US.UTF-8'
  
```

```
LC_ALL = 'en_US.UTF-8'
} // this has to be added only if you get an error saying UTF required is 8
but showing in ISO00009

tools {      maven 'Maven' // Ensure this matches the
name configured in Jenkins

}

stages

{

stage('
Checko
ut')

{

steps {

        git branch: 'master', url: 'https://github.com/shreya20m
/MavenAnsibleWebApp1CICD.git'

    }

    stage('Build') {

steps { sh 'mvn clean package' //
Run Maven build

    }

    }
```

```
    stage('Archive') {  
      steps { archiveArtifacts artifacts: 'target/*.war',  
fingerprint:true  
      }  
    }  
    stage('Deploy') {  
      steps { sh 'mvn clean package'  
sh 'ansible-playbook ansible/playbook.yml -i  
ansible/hosts.ini'  
      } } } }
```

Step 2: Jenkinsfile: To deploy **MavenAnsibleWebApp1.go** to the Jenkins home page and click on “**New Item**” enter the item name and select **Pipeline**

as the project type. Click “OK” .

New Item

Enter an item name

MavenAnsibleWebApp1-CICD

Select an item type

	Pipeline Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
	Freestyle project Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
	Maven project Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
	Multi-configuration project Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
	Folder Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
	Multibranch Pipeline Creates a set of Pipeline projects according to detected branches in one SCM repository.
	Organization Folder Creates a set of multibranch project subfolders by scanning for repositories.

OK

Step 3: In **MavenAnsibleWebApp-CICD**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of

MavenAnsibleWebApp. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the MavenAnsibleWebApp folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.

The screenshot shows the Jenkins 'Configure' page for a job named 'MavenAnsibleWebApp'. The 'SCM' section is set to 'Git'. The 'Repositories' section shows a single repository with the URL 'https://github.com/Samiksha201104/MavenAnsibleWebApp1.git' and 'GitHub Credentials' selected. The 'Advanced' options are expanded. At the bottom, there are 'Save' and 'Apply' buttons.

Step 4: Build in Jenkins

After clicking **Apply** and **Save**, go back to MavenAnsibleWebApp1-CICD in Jenkins and click **Build Now**. And resolve the issue.

```

+ ansible-playbook ansible/playbook.yml -i ansible/hosts.ini

PLAY [Deploy To Tomcat] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Copy WAR file to Tomcat] *****
changed: [localhost]

TASK [Restart Tomcat] *****
fatal: [localhost]: FAILED! => {"changed": false, "msg": "Could not find the requested service tomcat: host"}

PLAY RECAP *****
localhost      : ok=2   changed=1   unreachable=0   failed=1   skipped=0   rescued=0   ignored=0

```

Step 5: After making the changes, go back to **MavenAnsibleWebApp1-CICD** in Jenkins and click **Build Now**. The build should complete successfully without any errors.

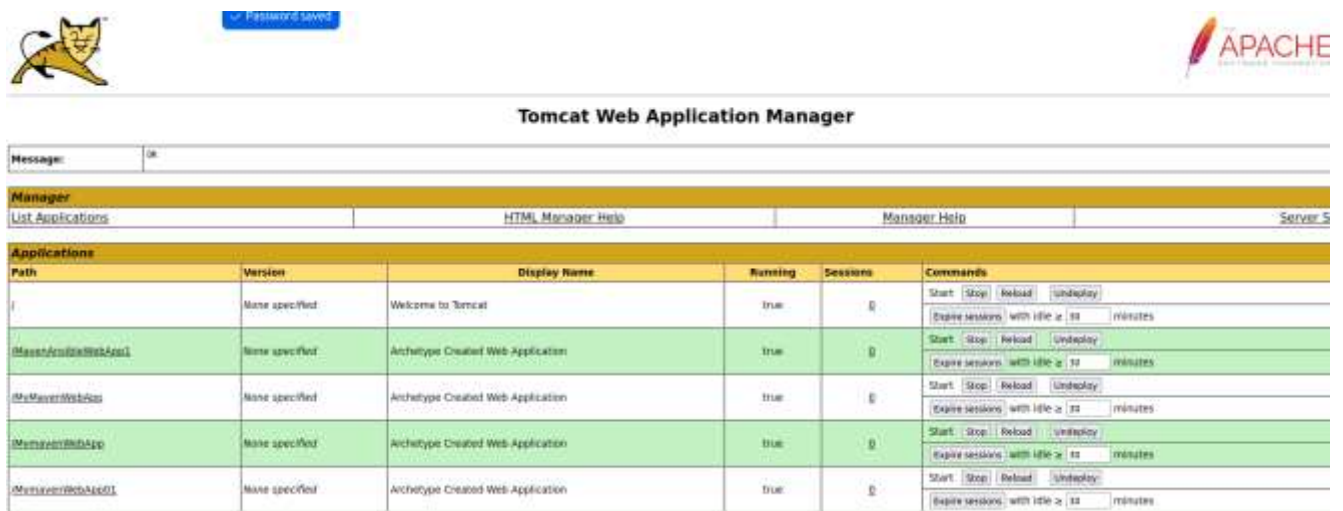
MavenAnsibleWebApp1-CICD - Stage View



Step 6 : Verify MavenAnsibleWebApp Deployment on Tomcat

start the server using `sh startup.sh`. After Tomcat starts successfully, we open the browser and access `http://localhost:9090` to verify the deployed application

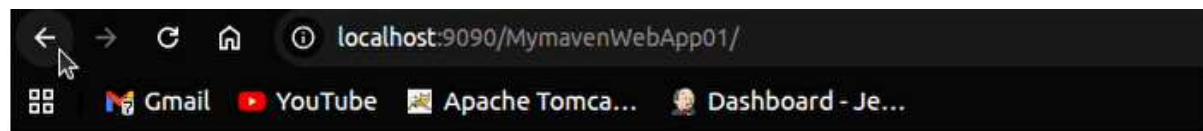
output.



The image shows the Tomcat Web Application Manager interface. At the top, there is a message box that says "Message: OK". Below this, there is a navigation bar with links: "List Applications", "HTML Manager Help", "Manager Help", and "Server Status". The main content area is titled "Applications" and contains a table with the following columns: Path, Version, Display Name, Running, Sessions, and Commands. The table lists several applications, including "Welcome to Tomcat", "Archetype Created Web Application", and "MymavenWebApp01". Each application has a "Start" button and a "Stop" button. The "MymavenWebApp01" application is highlighted in green.

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle > 30 minutes
/MymavenWebApp01	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle > 30 minutes
/MymavenWebApp02	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle > 30 minutes
/MymavenWebApp03	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle > 30 minutes
/MymavenWebApp04	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle > 30 minutes

Click on the project and to find the output.



The image shows a browser window with the address bar displaying "localhost:9090/MymavenWebApp01/". The browser has several tabs open, including "Gmail", "YouTube", "Apache Tomca...", and "Dashboard - Je...". The browser interface is dark-themed.

Samiksha P 1BI23CS189